

Java Generics — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Generics

UniqueClassName: Generics

1. Definition

Generics were introduced in **Java 5 (J2SE 5.0)** to provide **type-safe handling of classes, interfaces, and methods**.

Generics allow a class or method to work with a specific data type while providing **compile-time type checking**.

For example:

```
List<String> names = new ArrayList<>();
```

Here, String specifies that the list can contain only String objects.

Main Benefits

Benefit	Description
Type Safety	Prevents incompatible types from being added
Compile-Time Checking	Errors are detected during compilation
No Explicit Casting	Retrieved values usually don't require casting
Code Reusability	One generic class/method can work with multiple types
Better Readability	Clearly communicates the expected data type

2. Basic Generic Syntax

```
class ClassName<T> {  
    T value;  
  
    void setValue(T value) {  
        this.value = value;  
    }  
  
    T getValue() {
```

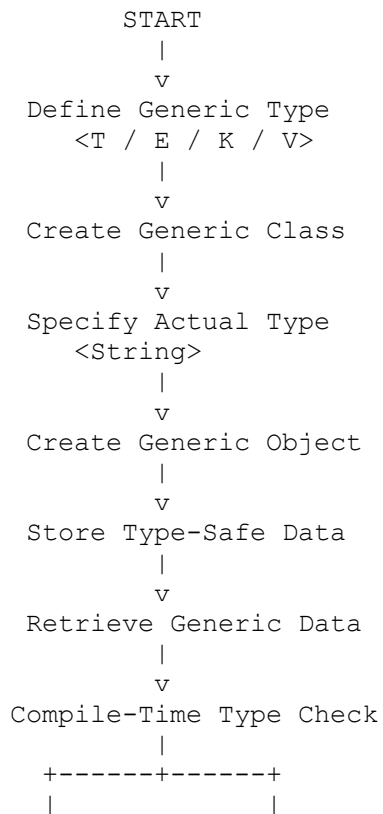
```
    return value;
  }
}
```

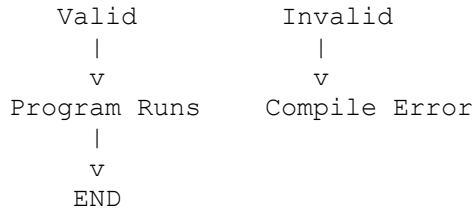
T is called a **type parameter**.

Common type-parameter naming conventions:

Parameter	Meaning
T	Type
E	Element
K	Key
V	Value
N	Number

3. Flowchart





4. Java Code Example — Generic Class

```
class Box<T> {  
  
    private T value;  
  
    public void setValue(T value) {  
        this.value = value;  
    }  
  
    public T getValue() {  
        return value;  
    }  
}  
  
public class Generics {  
  
    public static void main(String[] args) {  
  
        Box<String> stringBox = new Box<>();  
        stringBox.setValue("Java Generics");  
  
        Box<Integer> integerBox = new Box<>();  
        integerBox.setValue(100);  
  
        System.out.println("String: " + stringBox.getValue());  
        System.out.println("Integer: " + integerBox.getValue());  
    }  
}
```

Output

```
String: Java Generics  
Integer: 100
```

5. How the Example Works

Step 1 — Define Generic Class

```
class Box<T>
```

T represents an unknown type that will be specified when an object is created.

Step 2 — Create String Box

```
Box<String> stringBox = new Box<>();
```

Here:

T = String

Therefore, stringBox can store only String values.

```
stringBox.setValue("Java Generics");
```

Step 3 — Create Integer Box

```
Box<Integer> integerBox = new Box<>();
```

Here:

T = Integer

Therefore, the box stores Integer values.

```
integerBox.setValue(100);
```

6. Type-Safety Example

This is valid:

```
Box<String> box = new Box<>();
```

```
box.setValue("Hello");
```

But this is invalid:

```
Box<String> box = new Box<>();
```

```
box.setValue(100); // Compile-time error
```

Because:

```
Box<String>
|
+---- accepts String
|
```

+---- rejects Integer

The compiler detects the incorrect type before the program runs.

7. Generic Method Example

Generics can also be used with methods.

```
public class Generics {  
  
    public static <T> void printValue(T value) {  
        System.out.println("Value: " + value);  
    }  
  
    public static void main(String[] args) {  
  
        printValue("Java");  
        printValue(100);  
        printValue(25.5);  
    }  
}
```

Output

```
Value: Java  
Value: 100  
Value: 25.5
```

The same method works with different data types.

8. Generic Collection Example

Generics became especially important with the Java Collections Framework.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class Generics {  
  
    public static void main(String[] args) {  
  
        List<String> names = new ArrayList<>();  
  
        names.add("Anurag");  
        names.add("Rahul");  
    }  
}
```

```

names.add("Amit");

for (String name : names) {
    System.out.println(name);
}
}
}

```

Output

```

Anurag
Rahul
Amit

```

Without generics, older Java code commonly used:

```
List names = new ArrayList();
```

```
names.add("Anurag");
```

```
String name = (String) names.get(0);
```

With generics:

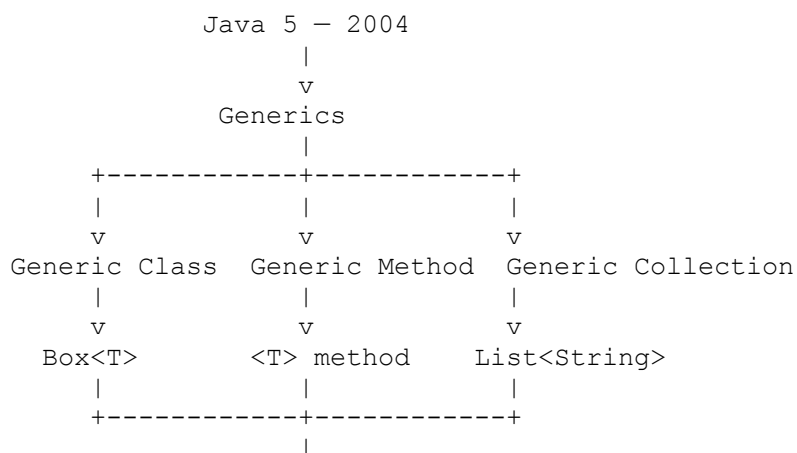
```
List<String> names = new ArrayList<>();
```

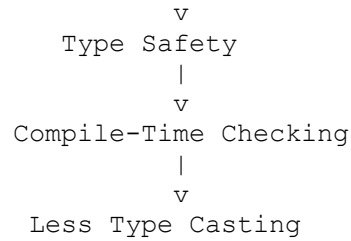
```
names.add("Anurag");
```

```
String name = names.get(0);
```

The explicit cast is no longer necessary.

9. Generics in Java 5 — Key Concept





In one sentence

Generics in Java 5 allow classes, interfaces, methods, and collections to operate on specified data types while providing compile-time type safety and reducing the need for explicit casting.

Java Enum — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: enum

UniqueClassName: Enums

1. Definition

An **enum (enumeration)** was introduced in **Java 5 (2004)**. It is a special Java type used to define a **fixed set of named constants**.

For example, days of the week, directions, months, status values, or user roles can be represented using an enum.

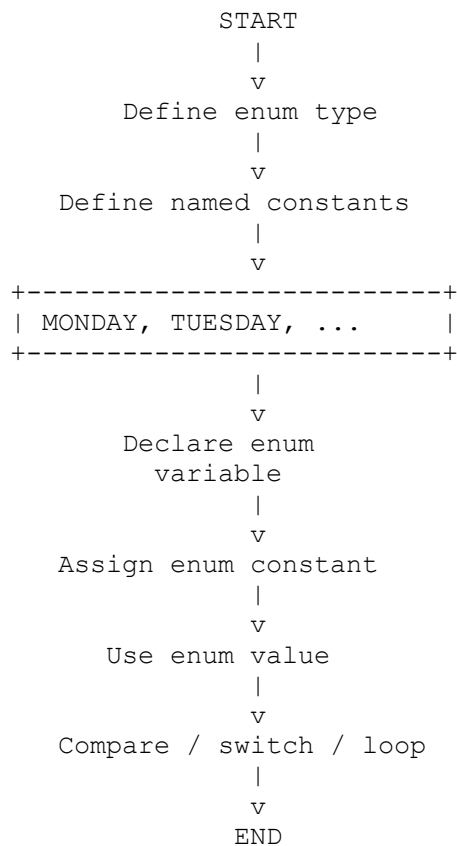
```
enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}
```

Instead of using arbitrary integer or string values, enums provide **type-safe named constants**.

2. Why Use Enum?

Without Enum	With Enum
<code>int status = 1;</code>	Status.ACTIVE
Meaning may be unclear	Meaning is clear
Invalid values can be assigned	Only defined constants are allowed
More error-prone	Type-safe
Constants may be scattered	Constants are grouped together

3. Flowchart



4. Basic Java Enum Example

```
enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}  
  
public class Enums {  
  
    public static void main(String[] args) {  
  
        Day today = Day.SUNDAY;  
  
        System.out.println("Today is: " + today);  
    }  
}
```

Output

Today is: SUNDAY

5. How It Works

The enum:

```
enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}
```

creates seven predefined constants.

We can declare a variable:

Day today;

and assign one of the defined constants:

```
today = Day.SUNDAY;
```

The following is **not allowed**:

```
Day today = "SUNDAY"; // Compile-time error
```

because "SUNDAY" is a String, not a Day.

6. Enum with switch

Enums work very well with switch.

```
enum Day {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
}

public class Enums {

    public static void main(String[] args) {

        Day day = Day.SATURDAY;

        switch (day) {

            case SATURDAY:
            case SUNDAY:
                System.out.println("Weekend");
                break;

            default:
                System.out.println("Weekday");
        }
    }
}
```

Output

Weekend

7. Enum with for Loop

The `values()` method returns all enum constants.

```
enum Direction {
    NORTH,
    SOUTH,
    EAST,
    WEST
}

public class Enums {

    public static void main(String[] args) {

        for (Direction direction : Direction.values()) {
            System.out.println(direction);
        }
    }
}
```

Output

```
NORTH
SOUTH
EAST
WEST
```

8. Enum with Fields and Constructor

Enums can also contain fields, constructors, and methods.

```
enum Status {

    ACTIVE(1),
    INACTIVE(0),
    PENDING(2);

    private int code;

    Status(int code) {
        this.code = code;
    }

    public int getCode() {
        return code;
    }
}

public class Enums {
```

```

public static void main(String[] args) {

    Status status = Status.ACTIVE;

    System.out.println("Status: " + status);
    System.out.println("Code: " + status.getCode());
}
}

```

Output

Status: ACTIVE
Code: 1

9. Important Enum Methods

Java provides several useful methods for enum types.

Method	Purpose
values()	Returns all enum constants
valueOf()	Converts a matching String to an enum constant
ordinal()	Returns the zero-based position of the constant
name()	Returns the name of the enum constant
toString()	Returns a string representation

Example:

```

enum Color {
    RED,
    GREEN,
    BLUE
}

public class Enums {

    public static void main(String[] args) {

        Color color = Color.GREEN;

        System.out.println(color.name());
        System.out.println(color.ordinal());
        System.out.println(Color.valueOf("BLUE"));
    }
}

```

```
        for (Color c : Color.values()) {  
            System.out.println(c);  
        }  
    }  
}
```

Output

```
GREEN  
1  
BLUE  
RED  
GREEN  
BLUE
```

10. Real-World Example

An application might have different order statuses:

```
enum OrderStatus {  
    CREATED,  
    PROCESSING,  
    SHIPPED,  
    DELIVERED,  
    CANCELLED  
}
```

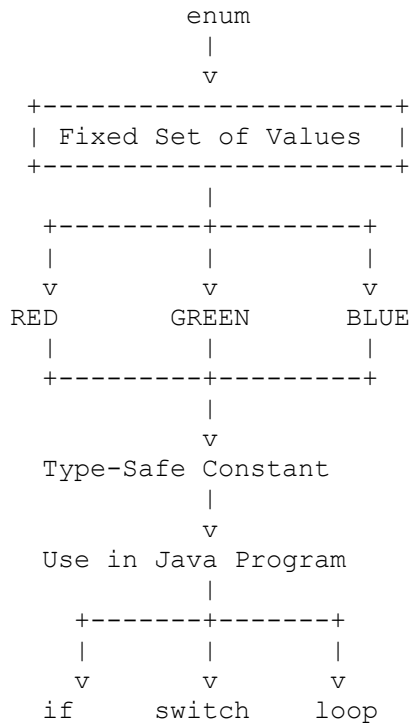
Usage:

```
public class Enums {  
  
    public static void main(String[] args) {  
  
        OrderStatus status = OrderStatus.SHIPPED;  
  
        if (status == OrderStatus.SHIPPED) {  
            System.out.println("Order has been shipped.");  
        }  
    }  
}
```

Output

```
Order has been shipped.
```

11. Enum Concept Flow



12. Key Points

- enum was introduced in **Java 5 (2004)**.
- It represents a **fixed set of constants**.
- Enum constants are **type-safe**.
- Enums can be used with if, switch, and loops.
- Enums can contain **fields, constructors, and methods**.
- `values()` returns all constants.
- `valueOf()` converts a string to the corresponding enum constant.
- `ordinal()` returns the constant's zero-based position.
- Enums are commonly used for **status, roles, directions, days, categories, and states**.

In one sentence:

Java enum is a special type introduced in Java 5 that provides a type-safe and organized way to represent a fixed set of named constants.

Java Annotations — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Annotations

UniqueClassName: Annotations

1. Definition

Annotations were introduced in **Java 5 (2004)**. An annotation is **metadata attached to Java code** that provides additional information about a class, method, field, parameter, or other program element.

Annotations begin with the @ symbol.

For example:

```
@Override
public String toString() {
    return "Java";
}
```

Here, @Override tells the Java compiler that the method is intended to override a method from a superclass.

Common Java Annotations

Annotation	Purpose
@Override	Indicates that a method overrides a superclass method
@Deprecated	Indicates that an element should no longer be used
@SuppressWarnings	Suppresses selected compiler warnings
@FunctionalInterface	Indicates that an interface is intended to be functional
@Retention	Specifies how long an annotation is retained
@Target	Specifies where an annotation can be applied

@Override, @Deprecated, and @SuppressWarnings are Java 5-era annotations.

@FunctionalInterface was added later in Java 8.

2. Why Annotations Are Used

Annotations allow developers and tools to attach **declarative metadata** to source code.

For example:

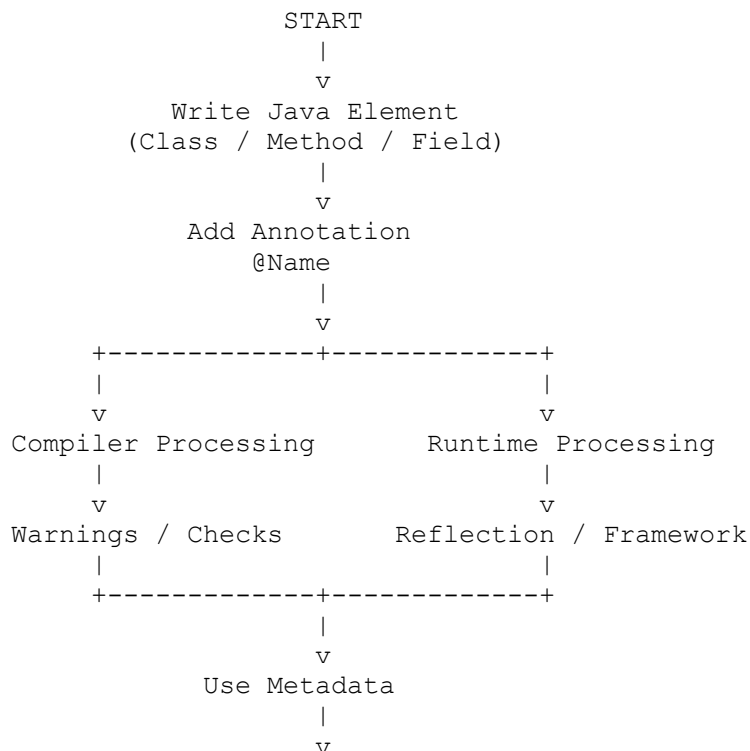
```
@Deprecated
public void oldMethod() {
}
```

The annotation communicates:

```
This method is deprecated
      ↓
Compiler / IDE can identify it
      ↓
Developer receives a warning
```

Annotations are widely used today by frameworks such as **Spring, JUnit, Jakarta EE, Hibernate, and many other Java libraries.**

3. Flowchart



END

4. Basic @Override Example

```
class Animal {  
    public void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
public class Annotations extends Animal {  
    @Override  
    public void sound() {  
        System.out.println("Dog makes a sound");  
    }  
  
    public static void main(String[] args) {  
        Annotations obj = new Annotations();  
        obj.sound();  
    }  
}
```

Output

Dog makes a sound

How It Works

The annotation:

@Override

tells the compiler:

"This method is intended to override a method from the parent class."

If the method is incorrectly written, the compiler can detect the problem.

For example:

```
@Override  
public void sounds() {  
}
```

If sounds() does not actually override a superclass method, the compiler reports an error.

5. @Deprecated Example

@Deprecated indicates that a method, class, or field should generally no longer be used.

```
public class Annotations {  
  
    @Deprecated  
    public static void oldMethod() {  
        System.out.println("Old method");  
    }  
  
    public static void main(String[] args) {  
        oldMethod();  
    }  
}
```

The compiler/IDE can warn the developer that oldMethod() is deprecated.

6. @SuppressWarnings Example

@SuppressWarnings can tell the compiler to suppress a particular warning.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class Annotations {  
  
    @SuppressWarnings("unchecked")  
    public static void main(String[] args) {  
  
        List list = new ArrayList();  
  
        list.add("Java");  
        list.add("Annotations");  
  
        System.out.println(list);  
    }  
}
```

Output

[Java, Annotations]

Here:

```
@SuppressWarnings("unchecked")
```

suppresses the specified compiler warning for the annotated element.

7. Creating a Custom Annotation

Java also allows developers to create their own annotations.

```
@interface Author {  
    String name();  
}  
  
@Author(name = "Anurag")  
public class Annotations {  
  
    public static void main(String[] args) {  
  
        System.out.println("Custom Annotation Example");  
    }  
}
```

Output

Custom Annotation Example

Here:

```
@interface Author
```

defines a custom annotation named Author.

It is then applied to the class:

```
@Author(name = "Anurag")
```

8. Custom Annotation with Reflection

Annotations become especially powerful when they are retained at runtime and inspected using reflection.

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;
```

```

@Retention(RetentionPolicy.RUNTIME)
@interface Author {
    String name();
}

@Author(name = "Anurag")
public class Annotations {

    public static void main(String[] args) {

        Class<Annotations> clazz = Annotations.class;

        Author author = clazz.getAnnotation(Author.class);

        System.out.println("Author: " + author.name());
    }
}

```

Output

Author: Anurag

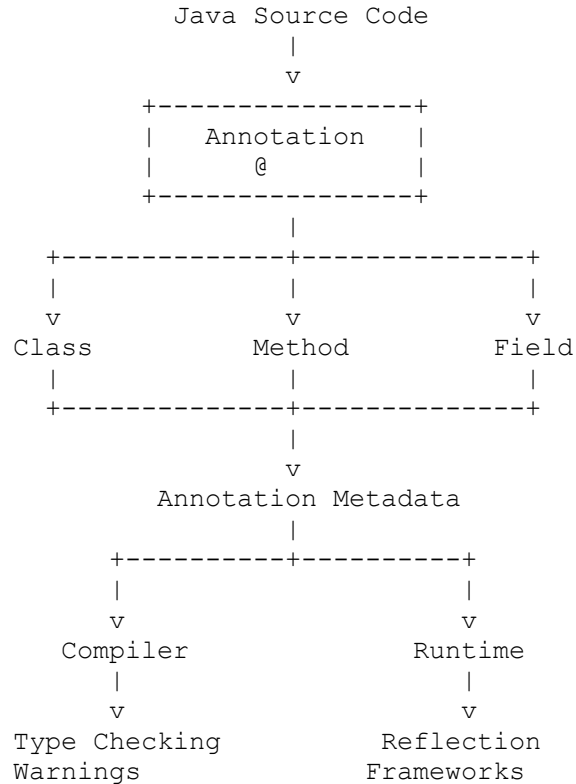
Execution Flow

```

@Author(name = "Anurag")
    |
    v
Annotation stored
    |
    v
RetentionPolicy.RUNTIME
    |
    v
Reflection
    |
    v
clazz.getAnnotation(...)
    |
    v
Read annotation
    |
    v
Print "Anurag"

```

9. Annotation Architecture



10. Important Annotation Concepts

1. Annotation Declaration

Defines a new annotation:

```
@interface Author {
    String name();
}
```

2. Annotation Usage

Applies an annotation:

```
@Author(name = "Anurag")
```

3. Retention

Determines how long annotation information is available.

```
@Retention(RetentionPolicy.RUNTIME)
```

Common retention policies:

Policy	Meaning
SOURCE	Available in source code only
CLASS	Stored in .class file
RUNTIME	Available at runtime through reflection

4. Target

Specifies where an annotation can be applied.

`@Target(ElementType.METHOD)`

For example, it can restrict an annotation to methods.

11. Real-World Example — Spring

Modern Java frameworks make extensive use of annotations.

For example, Spring uses:

`@Component`
`@Service`
`@Repository`
`@Controller`
`@Autowired`

A simplified example:

```
@Service
public class UserService {

    public void getUser() {
        System.out.println("Getting user");
    }
}
```

The framework reads the annotation and uses the metadata to determine how the class should be managed.

Conceptually:

@Service
|
v
Spring reads annotation
|
v
Identifies UserService
|
v
Creates/manages bean
|
v
Application can use UserService

12. Annotation vs Comment

Annotation	Comment
Starts with @	Starts with //, /*, or /**
Provides structured metadata	Provides human-readable notes
Can be processed by compiler/tools/frameworks	Normally ignored by compiler
Can affect compilation or runtime processing	Does not directly affect program behavior
Example: @Override	Example: // Override method

For example:

```
@Override  
public void display() {  
}
```

is meaningful to the compiler.

Whereas:

```
// This method overrides parent method  
public void display() {  
}
```

is simply a comment.

13. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Syntax:** @AnnotationName
- **Purpose:** Attach metadata to Java program elements.
- Can be applied to **classes, methods, fields, parameters, constructors**, etc.
- Built-in Java 5 annotations include **@Override**, **@Deprecated**, and **@SuppressWarnings**.
- Developers can create **custom annotations** using @interface.
- Runtime annotations can be accessed using **reflection**.
- Modern frameworks such as **Spring, JUnit, and Hibernate** heavily rely on annotations.

In one sentence

Java annotations are metadata constructs introduced in Java 5 that allow developers, compilers, tools, and frameworks to associate additional information with Java program elements using the @ syntax.

Java Autoboxing and Unboxing — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Autoboxing

UniqueClassName: AutoboxingUnboxing

1. Definition

Autoboxing was introduced in **Java 5 (2004)**. It is the **automatic conversion of a primitive data type into its corresponding wrapper class object**.

Unboxing is the reverse process: the **automatic conversion of a wrapper class object into its corresponding primitive data type**.

Primitive → Wrapper = Autoboxing

```
int number = 100;
```

```
Integer obj = number;
```

Conceptually:

int → Integer

Wrapper → Primitive = Unboxing

Integer obj = 100;

int number = obj;

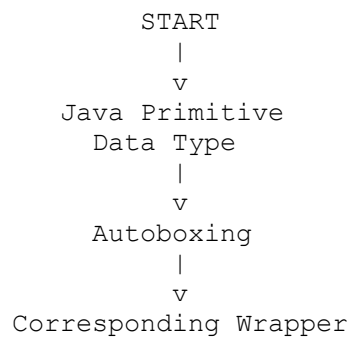
Conceptually:

Integer → int

2. Primitive and Wrapper Mapping

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

3. Flowchart



```
Class Object
  |
  v
Use as an Object
  |
  v
Unboxing
  |
  v
Java Primitive
Data Type
  |
  v
END
```

Detailed Flow

```
int number = 100
  |
  v
Autoboxing
  |
  v
Integer object = 100
  |
  |
  v
Unboxing
  |
  v
int value = object
```

4. Basic Autoboxing Example

```
public class AutoboxingUnboxing {

    public static void main(String[] args) {

        int number = 100;

        // Autoboxing
        Integer integerObject = number;

        System.out.println("Primitive: " + number);
        System.out.println("Wrapper: " + integerObject);
    }
}
```

Output

Primitive: 100

Wrapper: 100

The compiler automatically converts:

```
Integer integerObject = number;
```

into behavior conceptually similar to:

```
Integer integerObject = Integer.valueOf(number);
```

5. Basic Unboxing Example

```
public class AutoboxingUnboxing {  
    public static void main(String[] args) {  
        Integer integerObject = 200;  
  
        // Unboxing  
        int number = integerObject;  
  
        System.out.println("Wrapper: " + integerObject);  
        System.out.println("Primitive: " + number);  
    }  
}
```

Output

```
Wrapper: 200  
Primitive: 200
```

Conceptually, the compiler performs something similar to:

```
int number = integerObject.intValue();
```

6. Autoboxing + Unboxing Together

```
public class AutoboxingUnboxing {  
    public static void main(String[] args) {  
        // Primitive  
        int a = 10;  
  
        // Autoboxing
```

```

    Integer b = a;

    // Unboxing
    int c = b;

    System.out.println("a = " + a);
    System.out.println("b = " + b);
    System.out.println("c = " + c);
}
}

```

Output

```

a = 10
b = 10
c = 10

```

Flow:

```

int a
  |
  | Autoboxing
  v
Integer b
  |
  | Unboxing
  v
int c

```

7. Autoboxing with Collections

One of the most important uses of autoboxing is with the **Collections Framework**.

Collections store objects rather than primitive values.

Before Java 5, code commonly required explicit conversion:

```

List<Integer> numbers = new ArrayList<Integer>();

numbers.add(Integer.valueOf(10));
numbers.add(Integer.valueOf(20));

```

With autoboxing in Java 5:

```

import java.util.ArrayList;
import java.util.List;

```

```

public class AutoboxingUnboxing {

    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        System.out.println(numbers);
    }
}

```

Output

[10, 20, 30]

Here:

```
numbers.add(10);
```

automatically converts:

```

int
↓
Integer
↓
List<Integer>

```

8. Unboxing from a Collection

```

import java.util.ArrayList;
import java.util.List;

```

```

public class AutoboxingUnboxing {

    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        int first = numbers.get(0);
    }
}

```

```
        System.out.println("First number: " + first);
    }
}
```

Output

First number: 10

This:

```
int first = numbers.get(0);
```

automatically performs unboxing:

Integer
↓
int

9. Autoboxing with All Primitive Types

```
public class AutoboxingUnboxing {

    public static void main(String[] args) {

        Byte b = (byte) 10;
        Short s = (short) 20;
        Integer i = 30;
        Long l = 40L;
        Float f = 50.5f;
        Double d = 60.5;
        Character c = 'A';
        Boolean bool = true;

        System.out.println(b);
        System.out.println(s);
        System.out.println(i);
        System.out.println(l);
        System.out.println(f);
        System.out.println(d);
        System.out.println(c);
        System.out.println(bool);
    }
}
```

Output

10
20
30

40
50.5
60.5
A
true

10. Autoboxing in Expressions

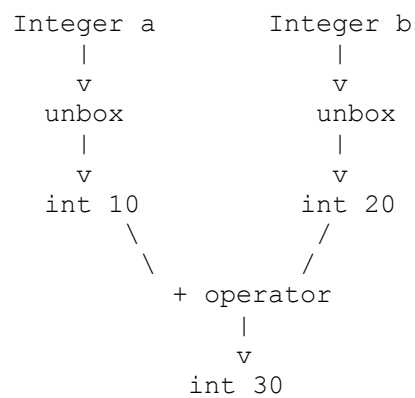
Java can also perform unboxing during arithmetic operations.

```
public class AutoboxingUnboxing {  
  
    public static void main(String[] args) {  
  
        Integer a = 10;  
        Integer b = 20;  
  
        int result = a + b;  
  
        System.out.println("Result: " + result);  
    }  
}
```

Output

Result: 30

Conceptually:



11. Important null Example

Wrapper objects can contain null, but primitives cannot.

```
public class AutoboxingUnboxing {  
    public static void main(String[] args) {  
        Integer number = null;  
        int value = number; // Runtime NullPointerException  
    }  
}
```

Why?

```
Integer number  
  |  
  v  
null  
  |  
  | Unboxing  
  v  
int  
  |  
  v  
Cannot convert null to primitive  
  |  
  v  
NullPointerException
```

Therefore, be careful when unboxing wrapper objects that might be null.

12. Before Java 5 vs Java 5

Before Java 5

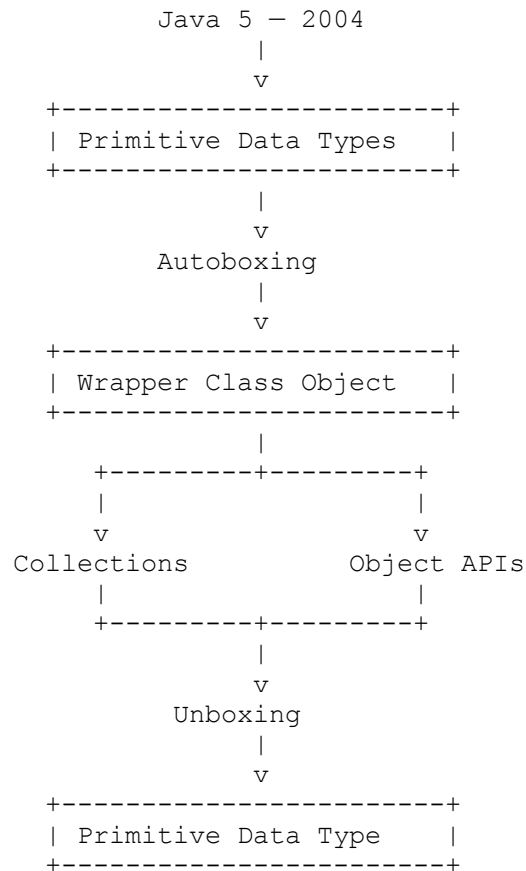
```
Integer number = Integer.valueOf(100);  
int value = number.intValue();
```

Java 5+

```
Integer number = 100;  
int value = number;
```

Java automatically performs the necessary conversions.

13. Complete Concept Flowchart



14. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Autoboxing:** Primitive → Wrapper
- **Unboxing:** Wrapper → Primitive
- Reduces the need for explicit conversion code.
- Particularly useful with **collections and generic types**.
- Wrapper classes are objects; primitives are not objects.
- Unboxing a null wrapper can cause `NullPointerException`.

In one sentence

Autoboxing and unboxing, introduced in Java 5, automatically convert between primitive types and their corresponding wrapper classes, making Java code—especially collections and generic code—simpler and more readable.

Java Varargs — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Varargs (Variable-Length Arguments)

UniqueClassName: Varargs

1. Definition

Varargs (variable-length arguments) were introduced in **Java 5 (2004)**. They allow a method to accept **zero or more arguments of the same type** without requiring the caller to create an array explicitly.

Varargs use three dots (...) after the data type:

dataType... parameterName

Example:

```
public static void add(int... numbers) {  
    // numbers is treated as an int[]  
}
```

The method can then be called with different numbers of arguments:

```
add();  
add(10);  
add(10, 20);  
add(10, 20, 30, 40);
```

2. Why Varargs Are Used

Without Varargs, you might need multiple overloaded methods:

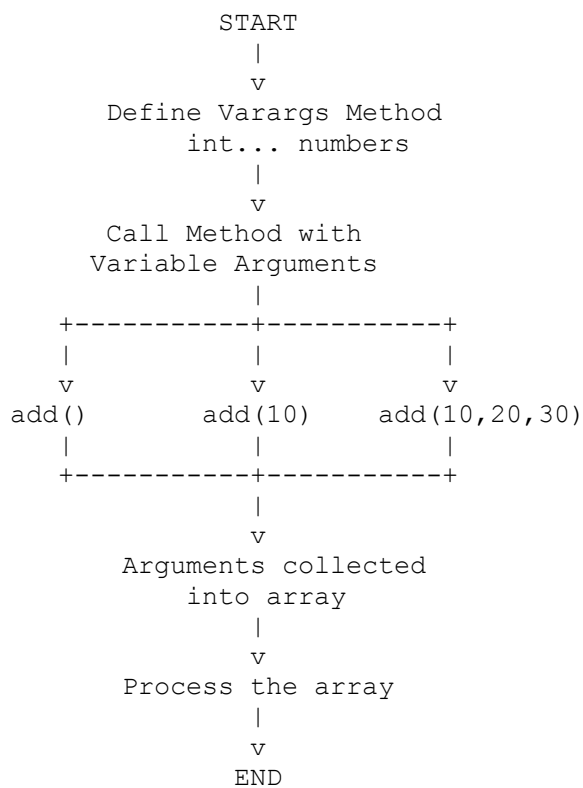
```
add(int a)
add(int a, int b)
add(int a, int b, int c)
add(int a, int b, int c, int d)
```

With Varargs:

```
add(int... numbers)
```

One method can handle all these cases.

3. Flowchart



Internal Concept

```
add(10, 20, 30)
  |
  v
int... numbers
  |
```

```
      v
int[] numbers = {10, 20, 30}
      |
      v
Process elements
```

4. Basic Java Varargs Example

```
public class Varargs {

    public static void printNumbers(int... numbers) {

        for (int number : numbers) {
            System.out.println(number);
        }
    }

    public static void main(String[] args) {

        printNumbers(10, 20, 30);
    }
}
```

Output

```
10
20
30
```

The parameter:

`int... numbers`

allows the method to receive multiple int values.

5. Varargs with Different Numbers of Arguments

```
public class Varargs {

    public static void printNumbers(int... numbers) {

        System.out.println("Number of arguments: " + numbers.length);

        for (int number : numbers) {
            System.out.println(number);
        }
    }
}
```

```

    }

    public static void main(String[] args) {

        printNumbers();
        printNumbers(10);
        printNumbers(10, 20);
        printNumbers(10, 20, 30, 40);
    }
}

```

Output

Number of arguments: 0

Number of arguments: 1
10

Number of arguments: 2
10
20

Number of arguments: 4
10
20
30
40

A Varargs parameter can therefore receive **zero, one, or many arguments**.

6. Varargs Example — Calculate Sum

```

public class Varargs {

    public static int sum(int... numbers) {

        int total = 0;

        for (int number : numbers) {
            total += number;
        }

        return total;
    }

    public static void main(String[] args) {

        System.out.println("Sum 1: " + sum(10, 20));
    }
}

```

```
        System.out.println("Sum 2: " + sum(10, 20, 30));
        System.out.println("Sum 3: " + sum(1, 2, 3, 4, 5));
    }
}
```

Output

```
Sum 1: 30
Sum 2: 60
Sum 3: 15
```

7. Varargs Are Internally Arrays

A Varargs parameter is treated as an array inside the method.

This:

```
public static void display(int... numbers) {
}
```

is conceptually similar to:

```
public static void display(int[] numbers) {
}
```

For example:

```
display(10, 20, 30);
```

is conceptually converted to something like:

```
display(new int[] {10, 20, 30});
```

Therefore, inside the method you can use:

```
numbers.length
numbers[0]
numbers[1]
```

and enhanced for loops.

8. Passing an Array to Varargs

You can also directly pass an array.

```

public class Varargs {

    public static void display(int... numbers) {

        for (int number : numbers) {
            System.out.println(number);
        }
    }

    public static void main(String[] args) {

        int[] values = {10, 20, 30, 40};

        display(values);
    }
}

```

Output

```

10
20
30
40

```

So Varargs can accept either:

Individual arguments

↓

10, 20, 30

or:

An array

↓

{10, 20, 30}

9. Varargs with Other Parameters

A method can have normal parameters before the Varargs parameter.

```

public class Varargs {

    public static void printStudent(String name, int... marks) {

        System.out.println("Student: " + name);

        for (int mark : marks) {

```

```
        System.out.println("Mark: " + mark);
    }
}

public static void main(String[] args) {

    printStudent("Anurag", 80, 85, 90);
}
}
```

Output

Student: Anurag
Mark: 80
Mark: 85
Mark: 90

Important Rule

The Varargs parameter **must be the last parameter**.

Correct:

```
void method(String name, int... values)
```

Incorrect:

```
void method(int... values, String name)
```

The second form causes a compilation error.

10. Varargs with Strings

Varargs are not limited to primitive types.

```
public class Varargs {

    public static void printNames(String... names) {

        for (String name : names) {
            System.out.println(name);
        }
    }

    public static void main(String[] args) {

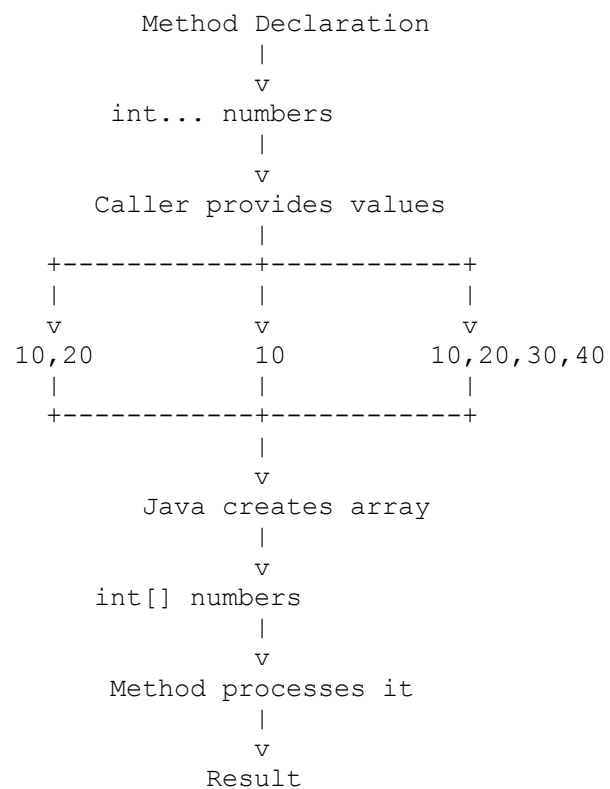
        printNames("Anurag", "Rahul", "Amit", "Vijay");
    }
}
```

```
}  
}
```

Output

Anurag
Rahul
Amit
Vijay

11. Varargs Flow — Complete



12. Varargs vs Array Parameter

Feature	Array Parameter	Varargs
Syntax	int[] values	int... values
Multiple individual arguments	✗	✓

Array accepted	✓	✓
Zero arguments	✗ unless array is supplied	✓
Introduced	Earlier Java	Java 5
Convenience	Lower	Higher

Example:

Array

```
void sum(int[] values)
```

Call:

```
sum(new int[]{10, 20, 30});
```

Varargs

```
void sum(int... values)
```

Call:

```
sum(10, 20, 30);
```

13. Important Rules of Varargs

Rule 1 — Only one Varargs parameter

A method cannot have multiple Varargs parameters.

Incorrect:

```
void test(int... a, String... b)
```

Rule 2 — Varargs must be last

Correct:

```
void test(String name, int... marks)
```

Incorrect:

```
void test(int... marks, String name)
```

Rule 3 — Varargs can accept zero arguments

```
test();
```

Rule 4 — Varargs are internally represented as arrays

```
int... numbers
```

behaves as an array inside the method.

14. Real-World Example

A logging utility could use Varargs:

```
public class Varargs {  
  
    public static void log(String message, Object... values) {  
  
        System.out.println(message);  
  
        for (Object value : values) {  
            System.out.println("Value: " + value);  
        }  
    }  
  
    public static void main(String[] args) {  
  
        log("User information", "Anurag", 1001, true);  
    }  
}
```

Output

```
User information  
Value: Anurag  
Value: 1001  
Value: true
```

This is useful when the number of values is not fixed.

15. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Feature:** Variable-Length Arguments

- **Syntax:** type... variable
- Allows a method to accept **zero or more arguments**.
- Varargs are internally handled as **arrays**.
- A method can have only **one Varargs parameter**.
- The Varargs parameter must be the **last parameter**.
- Varargs work with primitives, objects, and reference types.
- They reduce the need for multiple overloaded methods.

In one sentence

Varargs, introduced in Java 5, allow a method to accept a variable number of arguments of the same type using the ... syntax, with the arguments being handled internally as an array.

Java Enhanced for Loop — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Enhanced for Loop

UniqueClassName: EnhancedForLoop

1. Definition

The **Enhanced for loop**, also called the **for-each loop**, was introduced in **Java 5 (2004)**.

It provides a simple way to iterate through the elements of an **array** or an **Iterable collection** without explicitly managing an index or iterator.

Traditional for Loop

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

Enhanced for Loop

```
for (int number : numbers) {
    System.out.println(number);
}
```

The enhanced loop automatically visits each element.

2. Syntax

```
for (DataType variable : collectionOrArray) {  
    // statements  
}
```

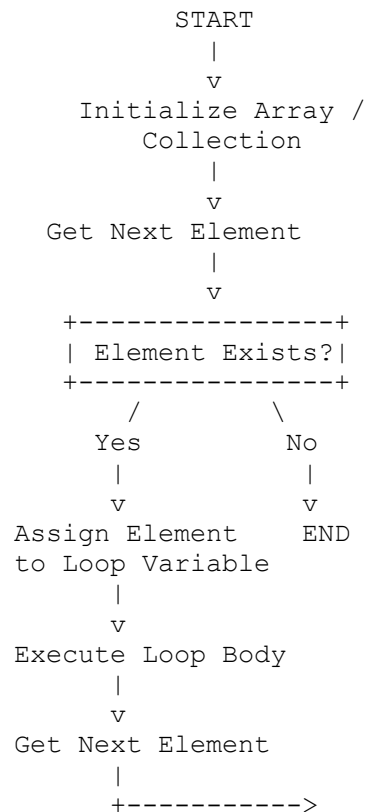
Example:

```
for (String name : names) {  
    System.out.println(name);  
}
```

Here:

- String → data type of each element
 - name → loop variable
 - names → array or Iterable collection
 - : → means "for each element in"
-

3. Flowchart



Example Flow

```

int[] numbers = {10, 20, 30}
    |
    v
    Enhanced for
    |
+-----+-----+
|                                     |
v                                     v
number=10                print 10
|
v
number=20
|
v
print 20
|
v
number=30
|
v
print 30
|
v
END

```

4. Basic Java Code Example

```
public class EnhancedForLoop {

    public static void main(String[] args) {

        int[] numbers = {10, 20, 30, 40, 50};

        for (int number : numbers) {
            System.out.println(number);
        }
    }
}
```

Output

10
20
30
40
50

The loop automatically processes every element of the array.

5. How It Works

Given:

```
int[] numbers = {10, 20, 30};
```

The enhanced loop:

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

works conceptually as:

First iteration:
number = 10

Second iteration:
number = 20

Third iteration:
number = 30

No more elements
↓
Loop terminates

6. Enhanced for Loop with String Array

```
public class EnhancedForLoop {  
    public static void main(String[] args) {  
        String[] names = {  
            "Anurag",  
            "Rahul",  
            "Amit",  
            "Vijay"  
        };  
        for (String name : names) {  
            System.out.println("Name: " + name);  
        }  
    }  
}
```

Output

Name: Anurag
Name: Rahul
Name: Amit
Name: Vijay

7. Enhanced for Loop with Collection

The enhanced for loop also works with collections implementing Iterable.

```
import java.util.ArrayList;
import java.util.List;

public class EnhancedForLoop {

    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();

        names.add("Anurag");
        names.add("Rahul");
        names.add("Amit");

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Output

Anurag
Rahul
Amit

8. Traditional for vs Enhanced for

Traditional for

```
int[] numbers = {10, 20, 30};

for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

Enhanced for

```
int[] numbers = {10, 20, 30};

for (int number : numbers) {
    System.out.println(number);
}
```

Comparison

Feature	Traditional for	Enhanced for
Introduced	Earlier Java	Java 5
Index required	Usually	No
Syntax	More verbose	Simpler
Array iteration	✓	✓
Iterable collection	Possible with index/iterator depending on collection	✓
Direct index access	✓	✗
Easy sequential traversal	Good	Excellent

9. Enhanced for with List<Integer>

```
import java.util.ArrayList;
import java.util.List;

public class EnhancedForLoop {

    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);

        for (Integer number : numbers) {
            System.out.println("Number = " + number);
        }
    }
}
```

```
}
```

Output

```
Number = 10  
Number = 20  
Number = 30  
Number = 40
```

Because Java 5 also introduced **generics and autoboxing**, this can be written more conveniently as:

```
List<Integer> numbers = new ArrayList<Integer>();  
  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);
```

10. Enhanced for with Objects

The loop can iterate over objects as well.

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}  
  
public class EnhancedForLoop {  
    public static void main(String[] args) {  
        Student[] students = {  
            new Student("Anurag"),  
            new Student("Rahul"),  
            new Student("Amit")  
        };  
  
        for (Student student : students) {  
            System.out.println(student.name);  
        }  
    }  
}
```

Output

Anurag
Rahul
Amit

11. Enhanced for Loop with Multidimensional Array

```
public class EnhancedForLoop {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
        for (int[] row : matrix) {  
            for (int value : row) {  
                System.out.print(value + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
1 2 3  
4 5 6  
7 8 9
```

12. Internal Working

For arrays, the enhanced for loop is essentially a convenient syntax for sequential array access.

This:

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

is conceptually similar to:

```
for (int i = 0; i < numbers.length; i++) {  
    int number = numbers[i];  
    System.out.println(number);  
}
```

For an Iterable collection, the compiler uses the collection's iterator mechanism conceptually similar to:

```
Iterator<String> iterator = names.iterator();  
  
while (iterator.hasNext()) {  
    String name = iterator.next();  
    System.out.println(name);  
}
```

So the enhanced loop hides the iteration details and makes sequential traversal easier to read.

13. Important Limitation — No Direct Index

Suppose:

```
int[] numbers = {10, 20, 30};
```

With a traditional loop, you can easily access the index:

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(i + " = " + numbers[i]);  
}
```

With enhanced for:

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

you directly get the **value**, not its index.

If you need the index, a traditional for loop is often more appropriate.

14. Important Limitation — Forward Sequential Traversal

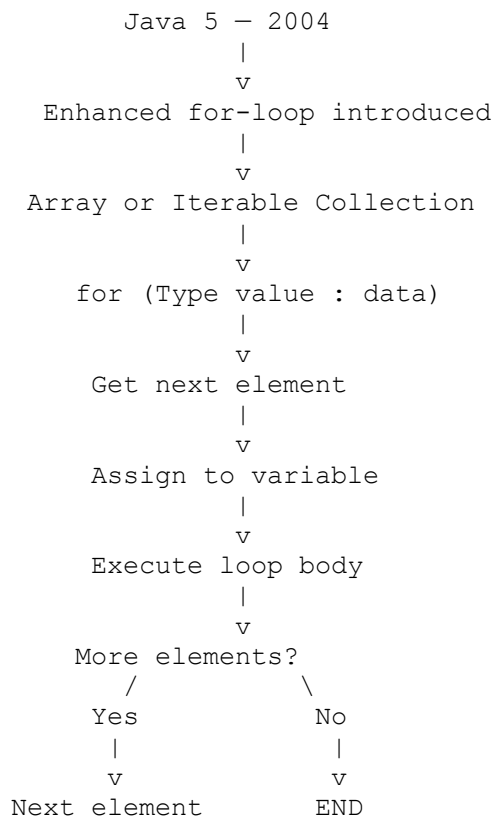
Enhanced for is primarily designed for straightforward traversal.

If you need operations such as:

- Access a specific index
- Move backward
- Skip to arbitrary positions
- Replace elements based on their index

a traditional loop or an appropriate iterator may be better.

15. Complete Flow



16. Real-World Example

Suppose an application needs to display employee names:

```

import java.util.Arrays;
import java.util.List;

public class EnhancedForLoop {

    public static void main(String[] args) {

        List<String> employees = Arrays.asList(
  
```

```
        "Anurag",  
        "Rahul",  
        "Amit",  
        "Vijay"  
    );  
  
    for (String employee : employees) {  
        System.out.println("Employee: " + employee);  
    }  
}  
}
```

Output

Employee: Anurag
Employee: Rahul
Employee: Amit
Employee: Vijay

17. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Feature:** Enhanced for loop / for-each loop
- Simplifies iteration over **arrays and Iterable collections**.
- Does not require explicit index management.
- Makes code shorter and more readable.
- Can iterate over primitives, objects, arrays, and collections.
- Does not directly provide the element's index.
- Best suited for **sequential traversal**.

In one sentence

The enhanced for loop introduced in Java 5 provides a simple and readable way to iterate sequentially over arrays and Iterable collections without explicitly managing indexes or iterators.

Java Static Import — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Static Import

UniqueClassName: StaticImport

1. Definition

Static import was introduced in **Java 5 (2004)**. It allows you to access **static members of another class directly by their simple names**, without repeatedly writing the class name.

Without static import:

```
System.out.println(Math.PI);  
System.out.println(Math.sqrt(25));
```

With static import:

```
import static java.lang.Math.PI;  
import static java.lang.Math.sqrt;
```

```
System.out.println(PI);  
System.out.println(sqrt(25));
```

In simple terms:

Without static import:

ClassName.staticMember

With static import:

staticMember

2. Why Static Import Is Used

Static import is useful when a program frequently uses static fields or methods from another class.

For example:

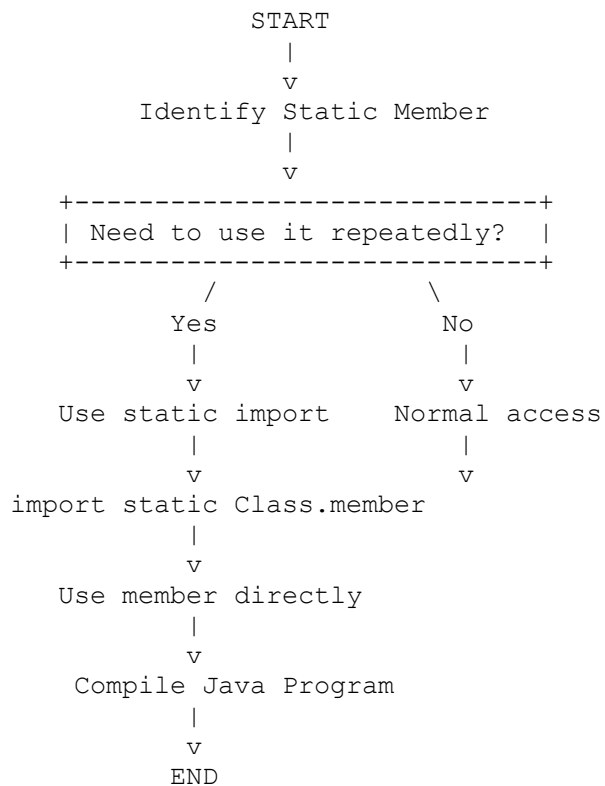
```
Math.PI  
Math.sqrt()  
Math.max()  
Math.min()
```

can be shortened to:

PI
sqrt()
max()
min()

when the corresponding static members are imported.

3. Flowchart



4. Basic Static Import Example

```
import static java.lang.Math.PI;

public class StaticImport {

    public static void main(String[] args) {

        System.out.println("Value of PI: " + PI);
    }
}
```

Output

Value of PI: 3.141592653589793

Normally, we would write:

```
System.out.println("Value of PI: " + Math.PI);
```

With static import:

```
System.out.println("Value of PI: " + PI);
```

5. Static Import of a Method

```
import static java.lang.Math.sqrt;

public class StaticImport {

    public static void main(String[] args) {

        double result = sqrt(25);

        System.out.println("Square Root: " + result);
    }
}
```

Output

Square Root: 5.0

Without static import:

```
double result = Math.sqrt(25);
```

With static import:

```
double result = sqrt(25);
```

6. Import Multiple Static Members

You can import multiple static members individually.

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;
```

```
import static java.lang.Math.pow;

public class StaticImport {

    public static void main(String[] args) {

        double radius = 5;

        double area = PI * pow(radius, 2);
        double squareRoot = sqrt(25);

        System.out.println("Area: " + area);
        System.out.println("Square Root: " + squareRoot);
    }
}
```

Output

Area: 78.53981633974483
Square Root: 5.0

7. Static Import Using *

You can import all accessible static members of a class using *.

```
import static java.lang.Math.*;

public class StaticImport {

    public static void main(String[] args) {

        System.out.println("PI: " + PI);
        System.out.println("Square Root: " + sqrt(16));
        System.out.println("Maximum: " + max(10, 20));
        System.out.println("Minimum: " + min(10, 20));
    }
}
```

Output

PI: 3.141592653589793
Square Root: 4.0
Maximum: 20
Minimum: 10

The syntax is:

```
import static package.ClassName.*;
```

8. Static Import of Your Own Class

Static import is not limited to Java's built-in classes.

Utility class

```
package utility;

public class Calculator {

    public static int add(int a, int b) {
        return a + b;
    }

    public static int multiply(int a, int b) {
        return a * b;
    }
}
```

Another class can use static import:

```
import static utility.Calculator.add;
import static utility.Calculator.multiply;

public class StaticImport {

    public static void main(String[] args) {

        System.out.println(add(10, 20));
        System.out.println(multiply(10, 20));
    }
}
```

Output

```
30
200
```

Without static import:

```
Calculator.add(10, 20);
Calculator.multiply(10, 20);
```

With static import:

```
add(10, 20);
multiply(10, 20);
```

9. Static Import of Static Variables

Static import can also be used with static fields.

```
class Constants {

    public static final double TAX_RATE = 18.0;
    public static final String COUNTRY = "India";
}
```

Use static import:

```
import static Constants.TAX_RATE;
import static Constants.COUNTRY;

public class StaticImport {

    public static void main(String[] args) {

        System.out.println("Country: " + COUNTRY);
        System.out.println("Tax Rate: " + TAX_RATE);
    }
}
```

Conceptually:

```
Constants.TAX_RATE
    |
    | static import
    v
TAX_RATE
```

10. Normal Import vs Static Import

Normal Import

Normal imports allow you to use a class without specifying its complete package name.

```
import java.util.ArrayList;

ArrayList<String> list = new ArrayList<String>();
```

You still need the class name:

ArrayList

Static Import

Static imports allow you to use static members without specifying their class name.

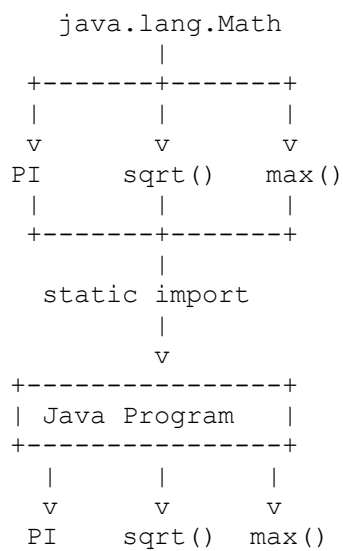
```
import static java.lang.Math.sqrt;
```

```
double result = sqrt(25);
```

Comparison

Normal Import	Static Import
Imports classes/interfaces	Imports static members
<code>import java.util.List;</code>	<code>import static java.lang.Math.PI;</code>
Use List	Use PI
Class name remains	Class name can be omitted

11. Static Import Flow



12. Static Import vs Static Member Access

Without Static Import

```
public class StaticImport {  
  
    public static void main(String[] args) {  
  
        System.out.println(Math.max(10, 20));  
        System.out.println(Math.min(10, 20));  
        System.out.println(Math.sqrt(25));  
    }  
}
```

With Static Import

```
import static java.lang.Math.max;  
import static java.lang.Math.min;  
import static java.lang.Math.sqrt;  
  
public class StaticImport {  
  
    public static void main(String[] args) {  
  
        System.out.println(max(10, 20));  
        System.out.println(min(10, 20));  
        System.out.println(sqrt(25));  
    }  
}
```

Both produce the same result.

13. Important Name Conflict

Static imports should be used carefully because two classes may have static members with the same name.

For example:

```
import static java.lang.Math.*;
```

If another imported class also provides a static method named `max()`, the compiler may encounter an ambiguity.

In such cases, use the class name explicitly:

```
Math.max(10, 20);
```

This improves clarity and removes ambiguity.

14. Complete Example

```
import static java.lang.Math.*;

public class StaticImport {

    public static void main(String[] args) {

        double radius = 10;

        double area = PI * pow(radius, 2);
        double circumference = 2 * PI * radius;

        System.out.println("Radius: " + radius);
        System.out.println("Area: " + area);
        System.out.println("Circumference: " + circumference);
        System.out.println("Square Root: " + sqrt(100));
        System.out.println("Maximum: " + max(50, 100));
        System.out.println("Minimum: " + min(50, 100));
    }
}
```

Output

```
Radius: 10.0
Area: 314.1592653589793
Circumference: 62.83185307179586
Square Root: 10.0
Maximum: 100
Minimum: 50
```

15. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Feature:** Static Import
- **Syntax:** import static package.ClassName.member;
- Allows static methods and fields to be accessed without the class name.
- Can import one static member:

```
import static java.lang.Math.PI;
```

- Can import multiple static members:

```
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;
```

- Can import all static members:

```
import static java.lang.Math.*;
```

- It improves brevity but should be used carefully when names may become ambiguous.

In one sentence

Static import, introduced in Java 5, allows static fields and methods to be referenced directly by their names without repeatedly qualifying them with the class name.

Java Concurrency Framework — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Concurrency Framework

UniqueClassName: ConcurrencyFramework

1. Definition

The **Concurrency Utilities Framework** was introduced in **Java 5 (2004)** through the `java.util.concurrent` package.

It provides high-level APIs for developing **multithreaded and concurrent applications** without having to manage every thread operation manually.

Before Java 5, developers commonly used:

```
Thread
Runnable
synchronized
wait()
notify()
notifyAll()
```

Java 5 introduced more powerful concurrency utilities such as:

- `ExecutorService`
- `Executors`

- Callable
 - Future
 - ConcurrentHashMap
 - BlockingQueue
 - CountdownLatch
 - Semaphore
 - AtomicInteger
 - Lock
 - ReentrantLock
-

2. Why the Concurrency Framework Was Introduced

Traditional thread management can become difficult when an application has many tasks.

Traditional approach

Create Thread
↓
Start Thread
↓
Execute Task
↓
Manage Thread
↓
Handle Synchronization
↓
Handle Results

Java 5 approach

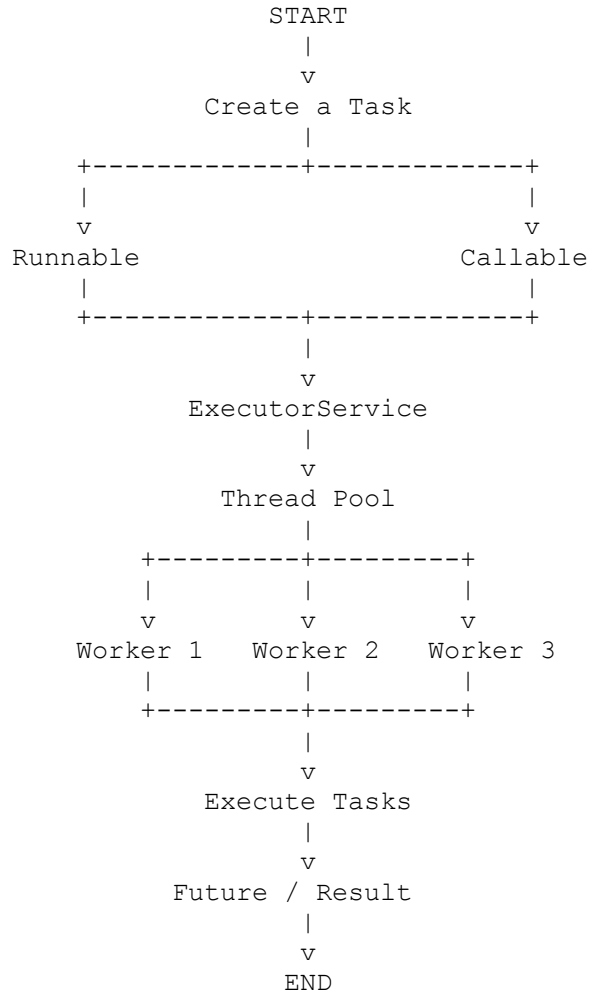
Submit Task
↓
ExecutorService
↓
Thread Pool
↓
Available Worker Thread
↓
Execute Task
↓
Return Result

This makes concurrent programming more manageable and scalable.

3. Major Components

Component	Purpose
Executor	Executes submitted tasks
ExecutorService	Manages task execution and thread lifecycle
Executors	Factory methods for creating executors/thread pools
Callable<V>	Represents a task that returns a result
Future<V>	Represents the result of an asynchronous computation
ScheduledExecutorService	Executes tasks after a delay or periodically
BlockingQueue	Thread-safe queue that can block during insertion/removal
ConcurrentHashMap	Thread-safe concurrent map
CountDownLatch	Allows threads to wait for other operations to complete
Semaphore	Controls access to a limited number of resources
AtomicInteger	Provides atomic integer operations
Lock	Provides explicit locking mechanisms

4. Flowchart



5. Basic ExecutorService Example

The simplest way to understand the Java 5 concurrency framework is through `ExecutorService`.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
```

```
public class ConcurrencyFramework {

    public static void main(String[] args) {

        ExecutorService executor =
            Executors.newFixedThreadPool(3);

        Runnable task = new Runnable() {
```

```

@Override
public void run() {
    System.out.println(
        "Task executed by: "
        + Thread.currentThread().getName()
    );
}
};

executor.submit(task);
executor.submit(task);
executor.submit(task);

executor.shutdown();
}
}

```

Possible Output

```

Task executed by: pool-1-thread-1
Task executed by: pool-1-thread-2
Task executed by: pool-1-thread-3

```

The exact thread names and execution order may vary.

6. How the Example Works

Step 1 — Create Thread Pool

```

ExecutorService executor =
    Executors.newFixedThreadPool(3);

```

This creates a thread pool containing up to **3 worker threads**.

```

ExecutorService
  |
  v
+-----+
| Thread Pool      |
+-----+
| Worker Thread 1  |
| Worker Thread 2  |
| Worker Thread 3  |
+-----+

```

Step 2 — Create Task

```
Runnable task = new Runnable() {  
  
    @Override  
    public void run() {  
        System.out.println("Executing task");  
    }  
};
```

Step 3 — Submit Task

```
executor.submit(task);
```

The executor schedules the task for execution.

Step 4 — Shutdown

```
executor.shutdown();
```

This tells the executor to stop accepting new tasks while allowing already submitted tasks to finish.

7. Callable and Future

One of the major improvements in Java 5 was the ability to submit a task that **returns a result**.

Runnable does not return a value.

Callable<V> can return a value.

```
import java.util.concurrent.Callable;  
import java.util.concurrent.ExecutorService;  
import java.util.concurrent.Executors;  
import java.util.concurrent.Future;  
  
public class ConcurrencyFramework {  
  
    public static void main(String[] args) throws Exception {  
  
        ExecutorService executor =  
            Executors.newSingleThreadExecutor();  
  
        Callable<Integer> task = new Callable<Integer>() {  
  
            @Override  
            public Integer call() {  
                return 10 + 20;  
            }  
        };  
    }  
};
```

```

        Future<Integer> future = executor.submit(task);

        Integer result = future.get();

        System.out.println("Result: " + result);

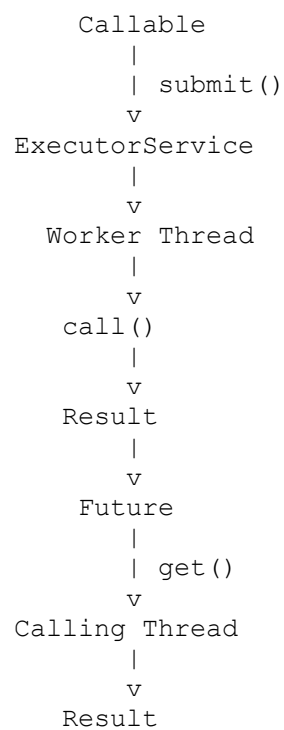
        executor.shutdown();
    }
}

```

Output

Result: 30

8. Callable → Future Flow



For example:

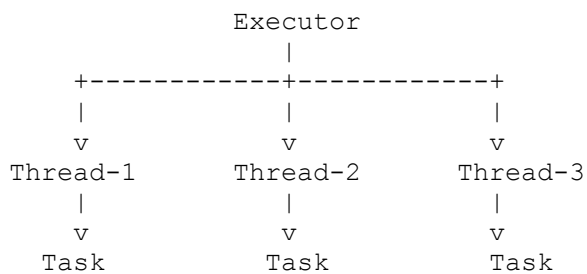
```

Callable<Integer>
    |
    v
    returns 30
    |
    v
Future<Integer>

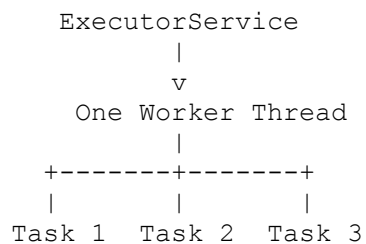
```

9. FixedThreadPool

```
ExecutorService executor =  
    Executors.newFixedThreadPool(3);
```



10. SingleThreadExecutor



Tasks are executed sequentially by one worker thread.

11. ScheduledExecutorService

Java 5 also introduced scheduled execution utilities.

```
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class ConcurrencyFramework {

    public static void main(String[] args) {

        ScheduledExecutorService scheduler =
            Executors.newScheduledThreadPool(1);

        scheduler.schedule(
            new Runnable() {

                @Override
                public void run() {
                    System.out.println("Task executed after delay");
                }

            },
            2,
            TimeUnit.SECONDS
        );

        scheduler.shutdown();
    }
}
```

Output after approximately 2 seconds

Task executed after delay

12. ConcurrentHashMap

Java 5 introduced ConcurrentHashMap for concurrent access to map data.

```
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrencyFramework {

    public static void main(String[] args) {

        ConcurrentHashMap<Integer, String> users =
```

```

        new ConcurrentHashMap<Integer, String>();

        users.put(1, "Anurag");
        users.put(2, "Rahul");
        users.put(3, "Amit");

        System.out.println(users);
    }
}

```

Output

{1=Anurag, 2=Rahul, 3=Amit}

ConcurrentHashMap is designed for concurrent access by multiple threads.

13. BlockingQueue

A BlockingQueue is useful for producer-consumer applications.

```

import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class ConcurrencyFramework {

    public static void main(String[] args)
        throws InterruptedException {

        BlockingQueue<String> queue =
            new ArrayBlockingQueue<String>(10);

        queue.put("Java");
        queue.put("Concurrency");
        queue.put("Framework");

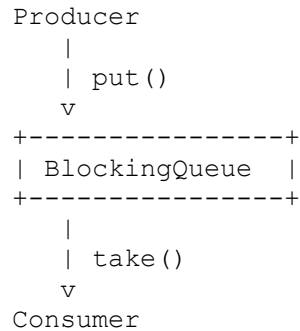
        System.out.println(queue.take());
        System.out.println(queue.take());
        System.out.println(queue.take());
    }
}

```

Output

Java
Concurrency
Framework

Conceptually:



14. CountdownLatch

CountDownLatch allows one or more threads to wait until a specified number of operations are completed.

```

import java.util.concurrent.CountDownLatch;

public class ConcurrencyFramework {

    public static void main(String[] args)
        throws InterruptedException {

        final CountDownLatch latch =
            new CountDownLatch(3);

        Runnable task = new Runnable() {

            @Override
            public void run() {

                System.out.println(
                    Thread.currentThread().getName()
                    + " completed"
                );

                latch.countDown();
            }
        };

        new Thread(task).start();
        new Thread(task).start();
        new Thread(task).start();

        latch.await();

        System.out.println("All tasks completed");
    }
}

```

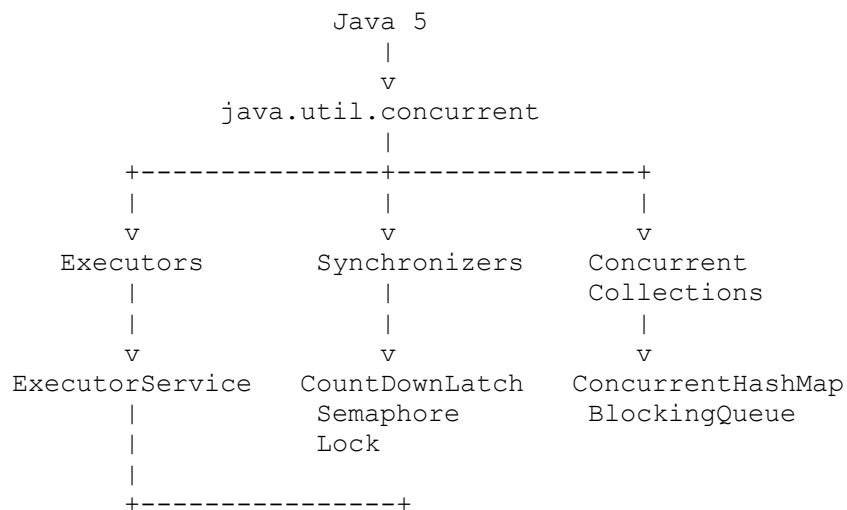
Possible Output

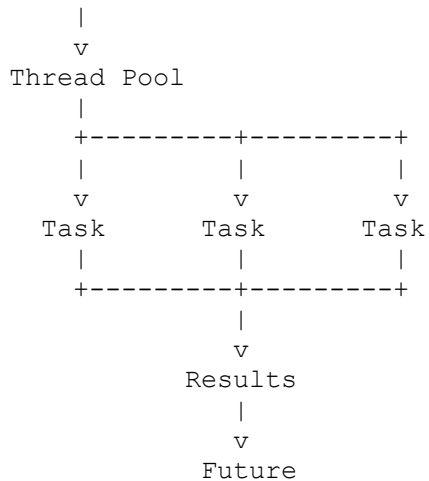
```
Thread-0 completed
Thread-1 completed
Thread-2 completed
All tasks completed
```

Flow:

```
CountDownLatch(3)
|
v
Task 1 done
|
v
count = 2
|
v
Task 2 done
|
v
count = 1
|
v
Task 3 done
|
v
count = 0
|
v
Waiting thread continues
```

15. Concurrency Framework Architecture

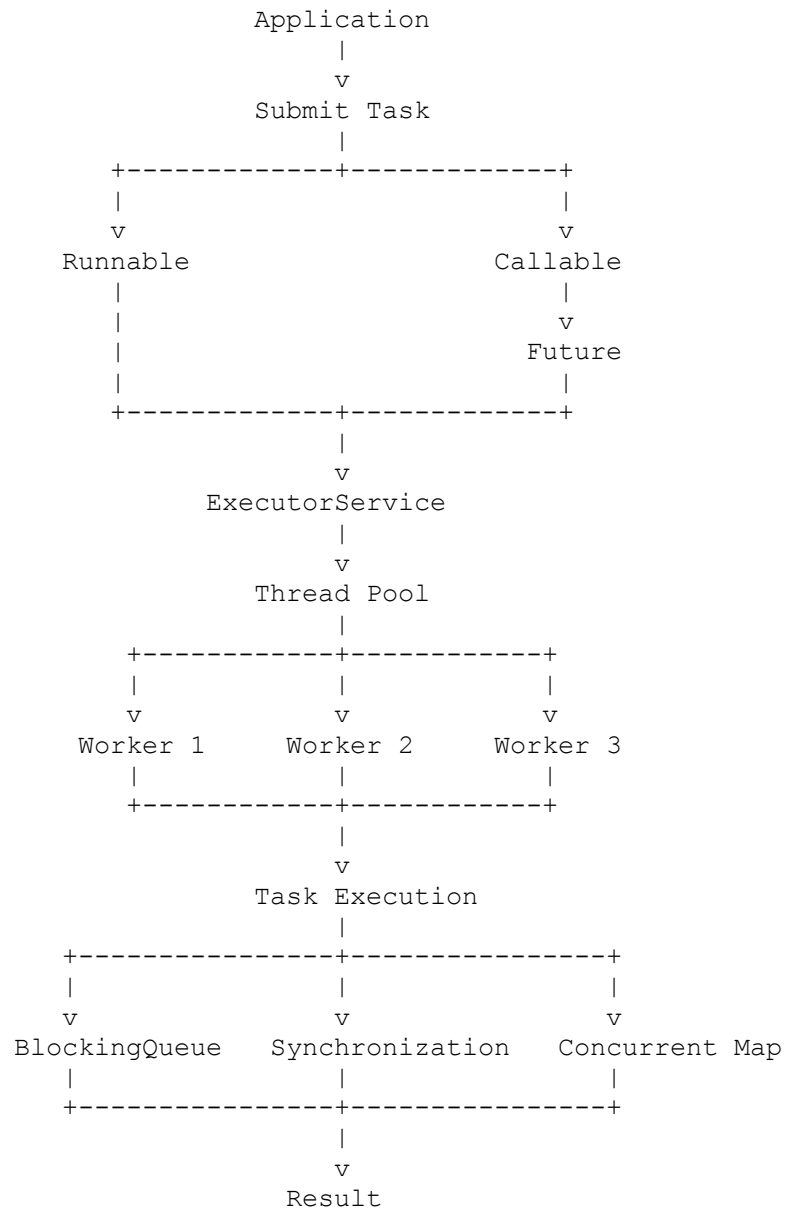




16. Major Benefits

Benefit	Explanation
Thread Pools	Reuse worker threads instead of creating new threads repeatedly
Task Management	Submit tasks without manually managing every thread
Result Handling	Callable and Future support asynchronous results
Synchronization	Utilities such as CountDownLatch, Semaphore, and Lock simplify coordination
Concurrent Collections	Provides thread-friendly collections
Scheduling	Allows delayed and periodic task execution
Scalability	Makes it easier to build applications with many concurrent tasks
Maintainability	Higher-level APIs reduce low-level thread-management code

17. Java 5 Concurrency Flow



18. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Main package:** java.util.concurrent
- Provides high-level support for **multithreaded and concurrent programming**.
- Important APIs include:
 - ExecutorService
 - Executors

- Callable
- Future
- ScheduledExecutorService
- BlockingQueue
- ConcurrentHashMap
- CountdownLatch
- Semaphore
- Lock
- Atomic classes such as AtomicInteger
- Thread pools help **reuse threads and manage tasks efficiently**.
- Callable + Future provide a way to execute tasks and obtain results.
- Concurrent collections provide safer data structures for multithreaded applications.

In one sentence

The Java 5 Concurrency Framework provides high-level utilities in `java.util.concurrent` for managing threads, executing asynchronous tasks, coordinating threads, scheduling work, and safely accessing shared data in concurrent applications.

Java ExecutorService — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: ExecutorService

UniqueClassName: ExecutorService

1. Definition

ExecutorService was introduced in **Java 5 (2004)** as part of the **`java.util.concurrent`** package.

It provides a high-level mechanism for **managing and executing asynchronous tasks using a pool of threads**.

Instead of creating and managing individual Thread objects manually, you submit tasks to an ExecutorService, which manages the worker threads for you.

Traditional Thread Model

```
Thread thread = new Thread(task);
thread.start();
```

ExecutorService Model

```
ExecutorService executor =  
    Executors.newFixedThreadPool(3);
```

```
executor.submit(task);
```

Conceptually:

```
Task  
↓  
ExecutorService  
↓  
Thread Pool  
↓  
Worker Thread  
↓  
Task Execution
```

2. Why ExecutorService Is Used

Managing individual threads can become difficult when an application has many tasks.

ExecutorService provides:

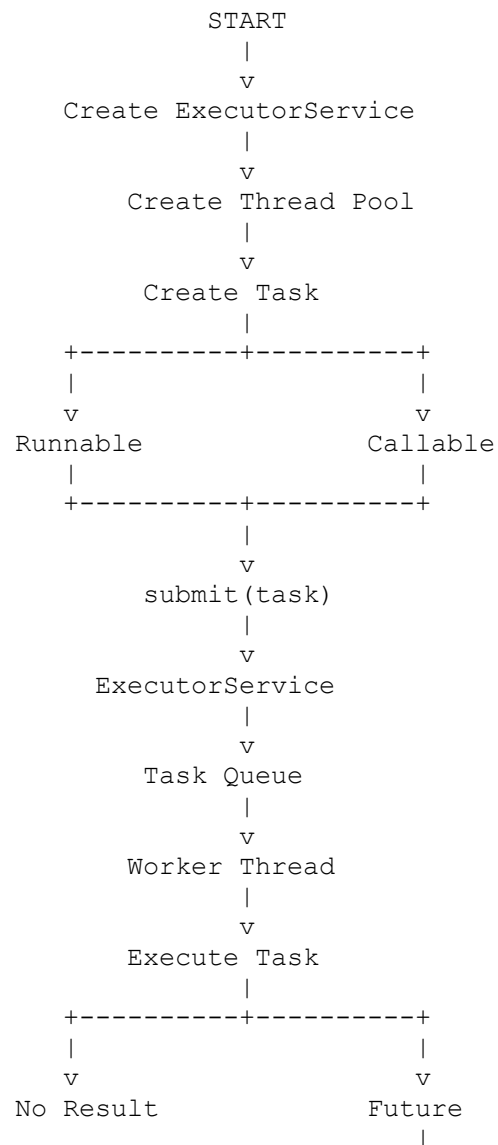
- Thread-pool management
 - Task submission
 - Asynchronous execution
 - Runnable task execution
 - Callable task execution
 - Future result handling
 - Executor shutdown
 - Controlled thread reuse
-

3. Important Methods

Method	Purpose
submit()	Submits a task for execution
execute()	Executes a Runnable task
shutdown()	Stops accepting new tasks
shutdownNow()	Attempts to stop currently executing tasks

isShutdown()	Checks whether executor has been shut down
isTerminated()	Checks whether all tasks have completed
awaitTermination()	Waits for termination
invokeAll()	Executes multiple Callable tasks
invokeAny()	Executes multiple tasks and returns one successful result

4. Flowchart



```

                                v
                                get()
                                |
                                v
shutdown()
    |
    v
    END

```

5. Basic ExecutorService Example

```

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorService {

    public static void main(String[] args) {

        java.util.concurrent.ExecutorService executor =
            Executors.newFixedThreadPool(3);

        Runnable task = new Runnable() {

            @Override
            public void run() {

                System.out.println(
                    "Task executed by: "
                    + Thread.currentThread().getName()
                );
            }
        };

        executor.submit(task);
        executor.submit(task);
        executor.submit(task);

        executor.shutdown();
    }
}

```

Possible Output

```

Task executed by: pool-1-thread-1
Task executed by: pool-1-thread-2
Task executed by: pool-1-thread-3

```

Note: The exact thread names and execution order can vary.

Because your requested class name is `ExecutorService`, the example uses the fully qualified `java.util.concurrent.ExecutorService` type to avoid a naming conflict between the class and the imported interface.

6. How It Works

Step 1 — Create Executor

```
java.util.concurrent.ExecutorService executor =  
    Executors.newFixedThreadPool(3);
```

This creates a thread pool with up to three worker threads.

```
ExecutorService  
    |  
    v  
+-----+  
|   Thread Pool   |  
+-----+  
| Worker Thread 1 |  
| Worker Thread 2 |  
| Worker Thread 3 |  
+-----+
```

Step 2 — Create a Task

```
Runnable task = new Runnable() {  
  
    @Override  
    public void run() {  
        System.out.println("Executing task");  
    }  
};
```

Step 3 — Submit the Task

```
executor.submit(task);
```

The task is handed to the executor rather than directly creating a new thread.

Step 4 — Shut Down

```
executor.shutdown();
```

The executor stops accepting new tasks but allows already submitted tasks to finish.

7. execute() vs submit()

ExecutorService supports both methods.

execute()

```
executor.execute(task);
```

Used for a Runnable when you don't need a Future result.

submit()

```
executor.submit(task);
```

Can accept Runnable or Callable and returns a Future.

Feature	execute()	submit()
Accepts Runnable	Yes	Yes
Accepts Callable	No	Yes
Returns Future	No	Yes
Result handling	No	Yes
Exception handling	Directly through executor/thread mechanism	Can be observed through Future

8. ExecutorService with Callable

A major advantage of ExecutorService is executing tasks that return results.

```
import java.util.concurrent.Callable;
import java.util.concurrent.Future;

public class ExecutorService {

    public static void main(String[] args)
        throws Exception {

        java.util.concurrent.ExecutorService executor =
            java.util.concurrent.Executors
                .newSingleThreadExecutor();

        Callable<Integer> task = new Callable<Integer>() {
```

```

        @Override
        public Integer call() {
            return 10 + 20;
        }
    };

    Future<Integer> future = executor.submit(task);

    Integer result = future.get();

    System.out.println("Result: " + result);

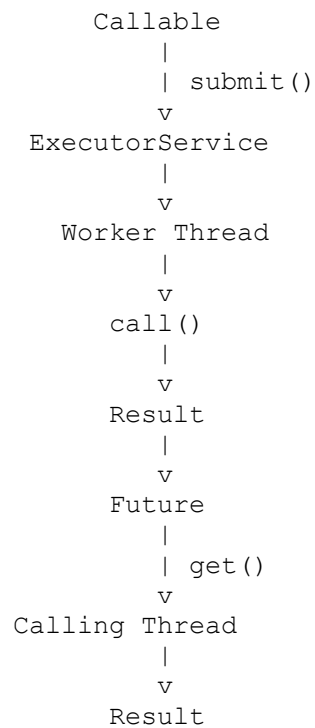
    executor.shutdown();
}
}

```

Output

Result: 30

9. Callable + Future Flow



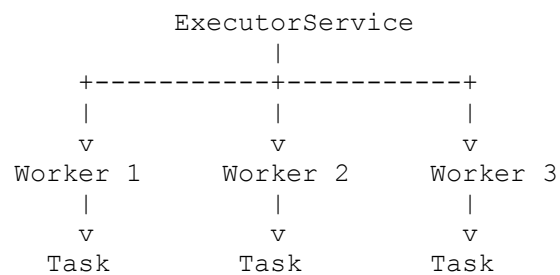
For example:

```
Callable<Integer>
  |
  v
call ()
  |
  v
 30
  |
  v
Future<Integer>
  |
  v
future.get ()
  |
  v
 30
```

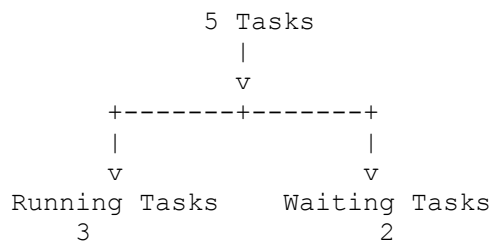
10. Fixed Thread Pool

```
java.util.concurrent.ExecutorService executor =
    Executors.newFixedThreadPool(3);
```

Conceptually:



If five tasks are submitted:

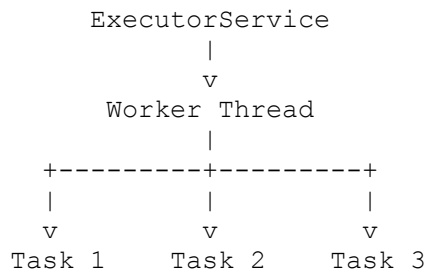


As worker threads finish, waiting tasks can be executed.

11. Single Thread Executor

```
java.util.concurrent.ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

Conceptually:



Only one worker thread executes tasks.

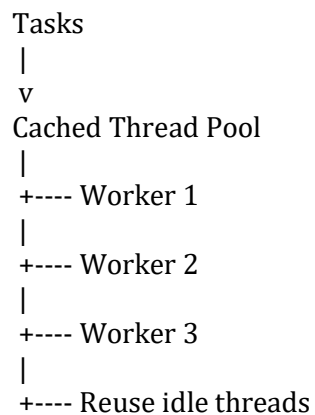
12. Cached Thread Pool

Java also provides:

```
java.util.concurrent.ExecutorService executor =  
    Executors.newCachedThreadPool();
```

It can create new threads as needed and reuse previously created idle threads when possible.

Conceptually:



13. Multiple Tasks Example

```
import java.util.concurrent.Executors;

public class ExecutorService {

    public static void main(String[] args) {

        java.util.concurrent.ExecutorService executor =
            Executors.newFixedThreadPool(2);

        for (int i = 1; i <= 5; i++) {

            final int taskNumber = i;

            executor.submit(new Runnable() {

                @Override
                public void run() {

                    System.out.println(
                        "Task " + taskNumber +
                        " executed by " +
                        Thread.currentThread().getName()
                    );
                }
            });
        }

        executor.shutdown();
    }
}
```

Possible Output

```
Task 1 executed by pool-1-thread-1
Task 2 executed by pool-1-thread-2
Task 3 executed by pool-1-thread-1
Task 4 executed by pool-1-thread-2
Task 5 executed by pool-1-thread-1
```

The exact ordering can vary because tasks execute concurrently.

14. Graceful Shutdown

A recommended pattern is to wait for submitted tasks to finish.

```
import java.util.concurrent.TimeUnit;
import java.util.concurrent.Executors;
```

```

public class ExecutorService {

    public static void main(String[] args)
        throws InterruptedException {

        java.util.concurrent.ExecutorService executor =
            Executors.newFixedThreadPool(2);

        executor.submit(new Runnable() {

            @Override
            public void run() {
                System.out.println("Task 1");
            }
        });

        executor.submit(new Runnable() {

            @Override
            public void run() {
                System.out.println("Task 2");
            }
        });

        executor.shutdown();

        if (executor.awaitTermination(
            10, TimeUnit.SECONDS)) {

            System.out.println("All tasks completed.");
        }
    }
}

```

Possible Output

Task 1
 Task 2
 All tasks completed.

15. shutdown() vs shutdownNow()

shutdown()

```
executor.shutdown();
```

Means:

Stop accepting new tasks



Finish submitted tasks



Terminate executor

shutdownNow()

```
executor.shutdownNow();
```

Attempts to stop currently executing tasks and returns tasks that were awaiting execution. It does not guarantee that every running task will stop immediately.

16. Real-World Example

Imagine an application needs to process five independent customer requests.

Instead of:

Request 1 → new Thread()

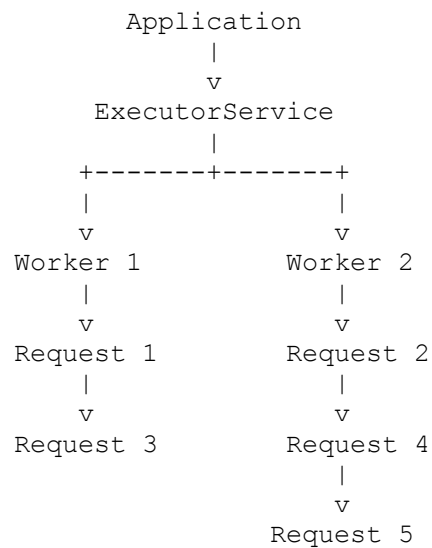
Request 2 → new Thread()

Request 3 → new Thread()

Request 4 → new Thread()

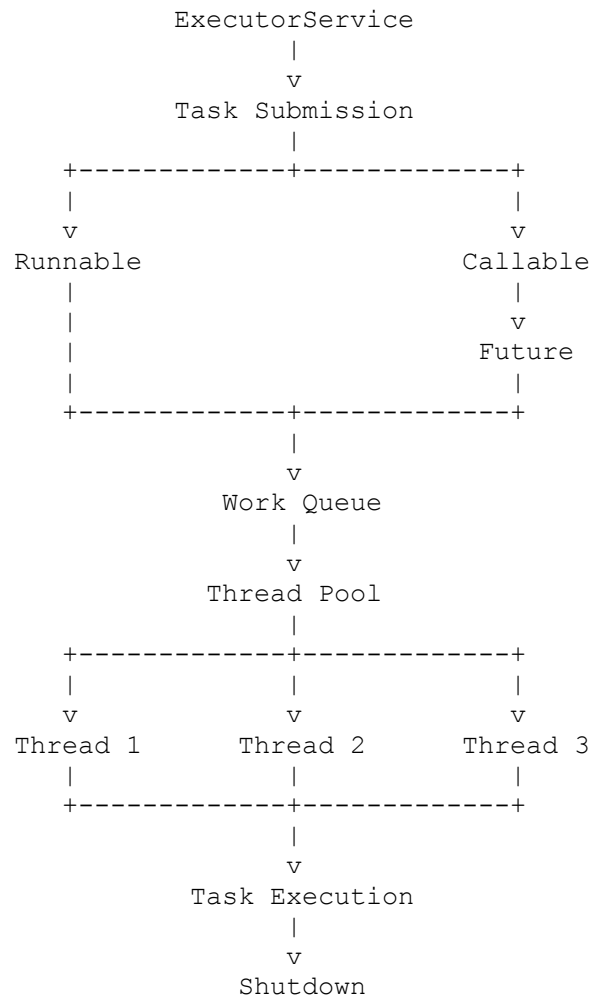
Request 5 → new Thread()

an executor can manage a fixed number of worker threads:



This allows threads to be **reused** rather than repeatedly created and destroyed.

17. ExecutorService Architecture



18. Important Interfaces and Classes

```
java.util.concurrent
|
+-- Executor
|
+-- ExecutorService
|
+-- ScheduledExecutorService
|
+-- Callable
|
+-- Future
|
+-- Executors
|
+-- ThreadPoolExecutor
```

The relationship is roughly:

```
Executor
|
v
ExecutorService
|
+---- submit()
+---- shutdown()
+---- invokeAll()
+---- invokeAny()
```

19. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Package:** java.util.concurrent
- ExecutorService is an interface for **managing asynchronous task execution**.
- It commonly works with **thread pools**.
- Tasks can be submitted using submit().
- Runnable is suitable for tasks without a return value.
- Callable is suitable for tasks that return a value.
- Future represents the result of an asynchronous computation.
- shutdown() provides graceful termination.
- shutdownNow() attempts to stop active execution and returns queued tasks.
- Thread pools improve **thread reuse, resource management, and scalability**.

In one sentence

ExecutorService, introduced in Java 5, provides a high-level framework for submitting, executing, managing, and shutting down concurrent tasks using managed worker threads and thread pools.

Java Callable / Future — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Callable / Future

UniqueClassName: CallableFuture

1. Definition

Callable and Future were introduced in **Java 5 (2004)** as part of the **java.util.concurrent** package.

They provide a mechanism for **executing a task asynchronously and obtaining its result later**.

Callable

Callable<V> represents a task that:

- can execute concurrently,
- **returns a result**,
- can throw checked exceptions.

Example:

```
Callable<Integer> task = new Callable<Integer>() {  
    @Override  
    public Integer call() {  
        return 10 + 20;  
    }  
};
```

Future

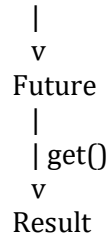
Future<V> represents the **result of an asynchronous computation**.

It allows you to:

- retrieve the result using get(),
- check whether the task is complete,
- cancel the task,
- check whether the task was cancelled.

Simple relationship:

```
Callable  
|  
| submit()  
v  
ExecutorService
```



2. Callable vs Runnable

Before Java 5, Runnable was commonly used for tasks.

```
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("Task executed");
    }
};
```

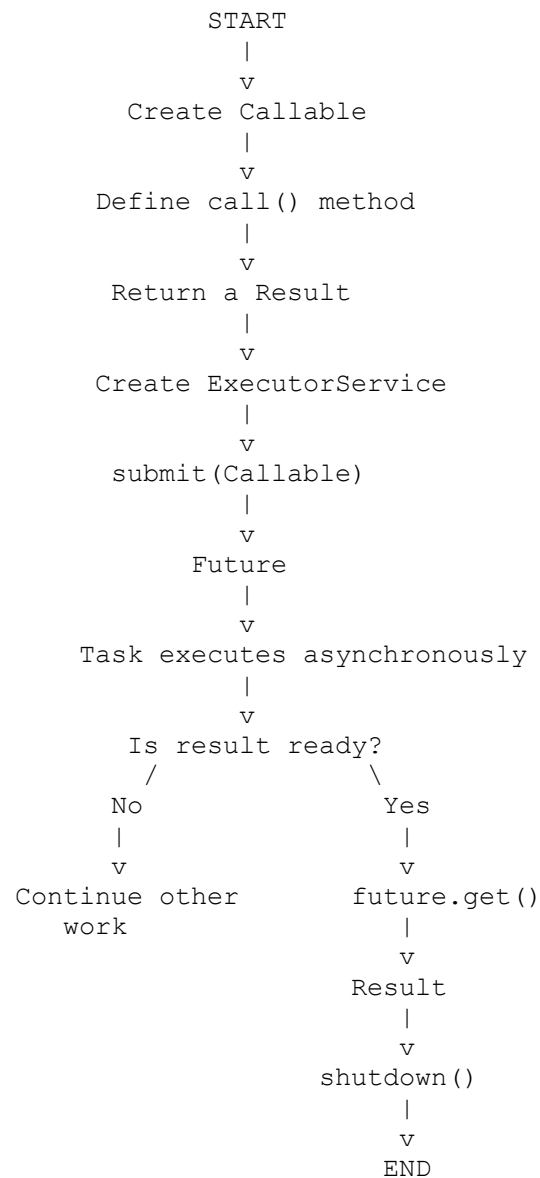
Runnable does not directly return a result.

Callable can return a result:

```
Callable<Integer> task = new Callable<Integer>() {
    @Override
    public Integer call() {
        return 100;
    }
};
```

Feature	Runnable	Callable<V>
Introduced	Java 1.0	Java 5
Method	run()	call()
Returns result	No	Yes
Throws checked exception	No	Yes
Used with ExecutorService	Yes	Yes
Result through Future	Generally no	Yes

3. Flowchart



4. Basic Java Code Example

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CallableFuture {
```

```

public static void main(String[] args)
    throws Exception {

    ExecutorService executor =
        Executors.newSingleThreadExecutor();

    Callable<Integer> task = new Callable<Integer>() {

        @Override
        public Integer call() {
            return 10 + 20;
        }
    };

    Future<Integer> future = executor.submit(task);

    Integer result = future.get();

    System.out.println("Result: " + result);

    executor.shutdown();
}

```

Output

Result: 30

5. How the Code Works

Step 1 — Create Executor

```

ExecutorService executor =
    Executors.newSingleThreadExecutor();

```

This creates an executor with one worker thread.

```

ExecutorService
  |
  v
+-----+
| Worker Thread |
+-----+

```

Step 2 — Create Callable

```

Callable<Integer> task = new Callable<Integer>() {

    @Override

```

```

    public Integer call() {
        return 10 + 20;
    }
};

```

The generic type:

```

Callable<Integer>
    |
    v
Returns Integer

```

The call() method returns:

30

Step 3 — Submit Callable

```
Future<Integer> future = executor.submit(task);
```

The executor schedules the task and immediately returns a Future.

```

Callable
    |
    | submit()
    v
ExecutorService
    |
    v
Worker Thread
    |
    v
Future<Integer>

```

Step 4 — Get Result

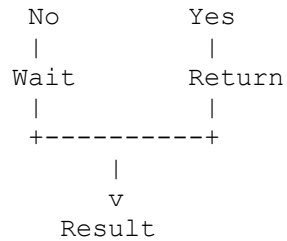
```
Integer result = future.get();
```

get() waits if necessary until the task has completed.

```

future.get()
    |
    v
Task completed?
    /      \

```



Step 5 — Shutdown

```
executor.shutdown();
```

This prevents new tasks from being submitted and allows submitted work to complete.

6. Asynchronous Execution Example

The important concept is that the task can execute on a worker thread while the calling thread continues other work.

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CallableFuture {

    public static void main(String[] args)
        throws Exception {

        ExecutorService executor =
            Executors.newSingleThreadExecutor();

        Callable<Integer> task = new Callable<Integer>() {

            @Override
            public Integer call()
                throws Exception {

                Thread.sleep(2000);

                return 100;
            }
        };

        Future<Integer> future =
            executor.submit(task);

        System.out.println("Task submitted.");
    }
}
```

```

        System.out.println(
            "Main thread can do other work.");

        Integer result = future.get();

        System.out.println(
            "Result: " + result);

        executor.shutdown();
    }
}

```

Possible Output

Task submitted.
Main thread can do other work.
Result: 100

The exact timing depends on the system.

7. Future Methods

Future provides several important methods.

Method	Purpose
get()	Waits for and returns the result
get(timeout, unit)	Waits up to a specified time
isDone()	Checks whether task is complete
isCancelled()	Checks whether task was cancelled
cancel()	Attempts to cancel the task

8. isDone() Example

```

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CallableFuture {

```

```

public static void main(String[] args)
    throws Exception {

    ExecutorService executor =
        Executors.newSingleThreadExecutor();

    Callable<Integer> task = new Callable<Integer>() {

        @Override
        public Integer call()
            throws Exception {

            Thread.sleep(2000);

            return 500;
        }
    };

    Future<Integer> future =
        executor.submit(task);

    System.out.println(
        "Is task done? " + future.isDone());

    Integer result = future.get();

    System.out.println(
        "Result: " + result);

    System.out.println(
        "Is task done? " + future.isDone());

    executor.shutdown();
}

```

Possible Output

```

Is task done? false
Result: 500
Is task done? true

```

9. Future with Timeout

Instead of waiting indefinitely, `get()` can use a timeout.

```

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

```

```

public class CallableFuture {

    public static void main(String[] args)
        throws Exception {

        ExecutorService executor =
            Executors.newSingleThreadExecutor();

        Callable<String> task = new Callable<String>() {

            @Override
            public String call()
                throws Exception {

                Thread.sleep(1000);

                return "Task completed";
            }
        };

        Future<String> future =
            executor.submit(task);

        String result =
            future.get(2, TimeUnit.SECONDS);

        System.out.println(result);

        executor.shutdown();
    }
}

```

Output

Task completed

If the task doesn't finish within the specified timeout, `get()` can throw `TimeoutException`.

10. Cancelling a Task

A Future can be used to request cancellation.

```

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CallableFuture {

    public static void main(String[] args)

```

```

        throws Exception {

    ExecutorService executor =
        Executors.newSingleThreadExecutor();

    Callable<String> task = new Callable<String>() {

        @Override
        public String call()
            throws Exception {

            Thread.sleep(5000);

            return "Completed";
        }
    };

    Future<String> future =
        executor.submit(task);

    Thread.sleep(1000);

    boolean cancelled =
        future.cancel(true);

    System.out.println(
        "Cancelled: " + cancelled);

    executor.shutdown();
}

```

Possible Output

Cancelled: true

`cancel(true)` requests interruption of the executing thread if the task has already started. Cancellation is cooperative; it does not guarantee immediate termination of arbitrary code.

11. Multiple Callable Tasks

`ExecutorService` can execute several `Callable` tasks.

```

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class CallableFuture {

    public static void main(String[] args)

```

```

        throws Exception {

    ExecutorService executor =
        Executors.newFixedThreadPool(3);

    Callable<Integer> task1 =
        new Callable<Integer>() {
            @Override
            public Integer call() {
                return 10;
            }
        };

    Callable<Integer> task2 =
        new Callable<Integer>() {
            @Override
            public Integer call() {
                return 20;
            }
        };

    Callable<Integer> task3 =
        new Callable<Integer>() {
            @Override
            public Integer call() {
                return 30;
            }
        };

    Future<Integer> future1 =
        executor.submit(task1);

    Future<Integer> future2 =
        executor.submit(task2);

    Future<Integer> future3 =
        executor.submit(task3);

    System.out.println(future1.get());
    System.out.println(future2.get());
    System.out.println(future3.get());

    executor.shutdown();
}
}

```

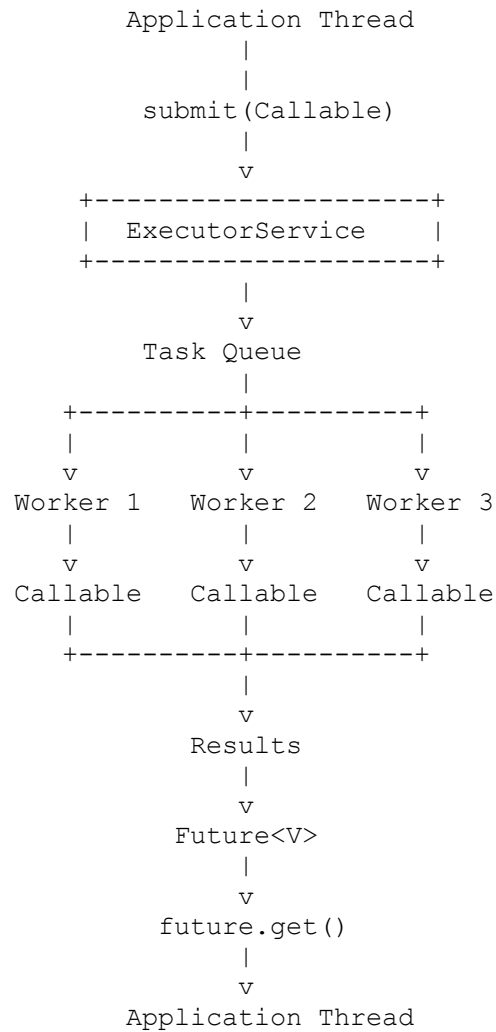
Output

```

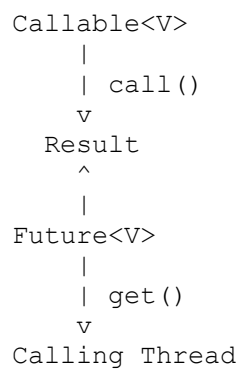
10
20
30

```

12. Complete Architecture



13. Callable / Future Relationship



A useful way to remember it:

Callable = "The task that produces the result"

Future = "The handle through which I get that result later"

14. Runnable vs Callable vs Future

Feature	Runnable	Callable	Future
Introduced	Java 1.0	Java 5	Java 5
Represents	Task	Result-producing task	Async computation result
Main method	run()	call()	get()
Returns value	No	Yes	Retrieves value
Checked exceptions	No	Yes	N/A
Cancellation	Through executor/thread mechanisms	Through returned Future	Yes
Used with ExecutorService	Yes	Yes	Returned by submission

15. Real-World Example

Suppose an application needs to calculate a customer's account balance.

```
Application
|
v
Submit balance calculation
|
v
Callable<Double>
|
v
ExecutorService
|
v
Worker Thread
|
```

```

    v
Calculate balance
    |
    v
Future<Double>
    |
    v
future.get()
    |
    v
Balance returned
```

Example:

```
Callable<Double> task = new Callable<Double>() {

    @Override
    public Double call() {

        double balance = 25000.50;

        return balance;
    }
};

Future<Double> future =
    executor.submit(task);

Double balance = future.get();
```

16. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Package:** java.util.concurrent
- Callable<V> represents a task that **returns a value**.
- Callable uses the call() method.
- Future<V> represents the **pending or completed result** of an asynchronous task.
- Future.get() retrieves the result.
- Future.isDone() checks completion.
- Future.cancel() attempts cancellation.
- Future.get(timeout, unit) supports timeout-based waiting.
- Callable can throw checked exceptions.
- Callable and Future are commonly used together with ExecutorService.

In one sentence

Callable and Future, introduced in Java 5, provide a powerful mechanism for executing result-producing tasks asynchronously and retrieving, monitoring, timing out, or cancelling their results through a Future.

Java Atomic Types — Java 5 (2004)

Java Version: 5

Year: 2004

Feature: Atomic Types

UniqueClassName: AtomicTypes

1. Definition

Atomic types were introduced in **Java 5 (2004)** as part of the `java.util.concurrent.atomic` package.

They provide **thread-safe operations on individual variables without requiring traditional synchronization such as synchronized blocks or methods.**

The most commonly used atomic classes introduced in Java 5 include:

- `AtomicInteger`
- `AtomicLong`
- `AtomicBoolean`
- `AtomicReference`
- `AtomicIntegerArray`
- `AtomicLongArray`
- `AtomicReferenceArray`

An atomic operation is performed as a **single, indivisible operation** from the perspective of other threads.

2. Why Atomic Types Are Needed

Consider this code:

```
counter++;
```

It looks like one operation, but internally it is approximately:

Read counter
↓
Add 1
↓
Write counter

With multiple threads, another thread can modify counter between these steps.

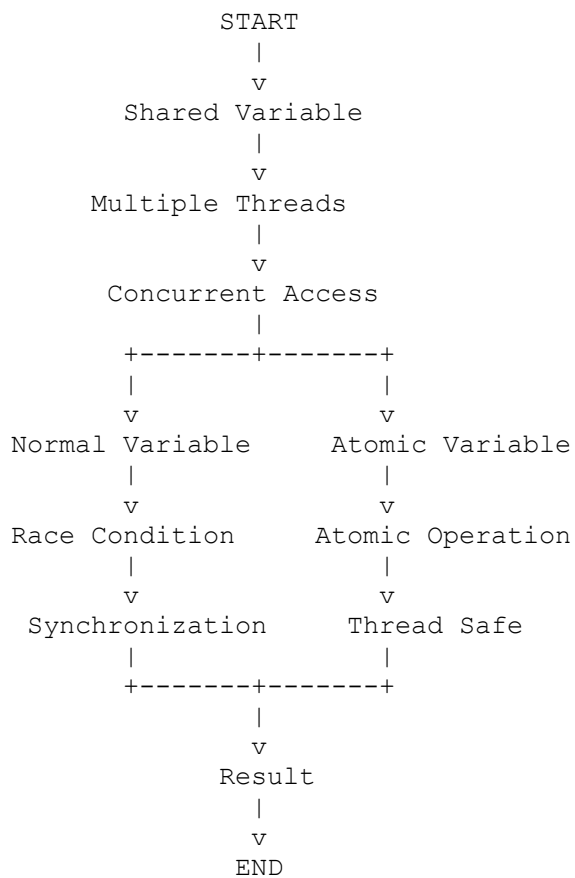
This can cause a **race condition**.

Atomic classes provide operations such as:

`counter.incrementAndGet();`

which performs the update atomically.

3. Flowchart



4. Basic AtomicInteger Example

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicTypes {

    public static void main(String[] args) {

        AtomicInteger counter =
            new AtomicInteger(0);

        counter.incrementAndGet();
        counter.incrementAndGet();

        System.out.println("Counter: "
            + counter.get());
    }
}
```

Output

Counter: 2

5. Important AtomicInteger Methods

Method	Purpose
get()	Gets the current value
set(value)	Sets a new value
incrementAndGet()	Increments and returns the new value
getAndIncrement()	Returns current value, then increments
decrementAndGet()	Decrements and returns the new value
getAndDecrement()	Returns current value, then decrements
addAndGet(value)	Adds value and returns new value
getAndAdd(value)	Returns current value, then adds
compareAndSet(expected, update)	Updates only if current value equals expected

6. Atomic Increment Example

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicTypes {

    public static void main(String[] args) {

        AtomicInteger counter =
            new AtomicInteger(10);

        int result =
            counter.incrementAndGet();

        System.out.println("New value: " + result);
    }
}
```

Output

New value: 11

7. `getAndIncrement()` vs `incrementAndGet()`

This distinction is important.

`incrementAndGet()`

```
AtomicInteger counter =
    new AtomicInteger(10);

int value = counter.incrementAndGet();
```

Result:

```
value = 11
counter = 11
```

`getAndIncrement()`

```
AtomicInteger counter =
    new AtomicInteger(10);

int value = counter.getAndIncrement();
```

Result:

value = 10
counter = 11

Flow

incrementAndGet()

|

v

Increment

|

v

Return NEW value

getAndIncrement()

|

v

Return OLD value

|

v

Increment

8. Multithreading Example

Atomic types are especially useful when multiple threads modify the same value.

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicTypes {

    public static void main(String[] args)
        throws InterruptedException {

        final AtomicInteger counter =
            new AtomicInteger(0);

        Thread thread1 = new Thread(
            new Runnable() {

                @Override
                public void run() {

                    for (int i = 0; i < 1000; i++) {
                        counter.incrementAndGet();
                    }
                }
            }
        );

        Thread thread2 = new Thread(
            new Runnable() {
```

```

        @Override
        public void run() {

            for (int i = 0; i < 1000; i++) {
                counter.incrementAndGet();
            }
        }
    };

    thread1.start();
    thread2.start();

    thread1.join();
    thread2.join();

    System.out.println(
        "Final Counter: " + counter.get()
    );
}

```

Output

Final Counter: 2000

Both threads safely increment the same AtomicInteger.

9. Normal int vs AtomicInteger

Consider:

```
int counter = 0;
```

```
counter++;
```

The increment is not an atomic operation.

Conceptually:

```

counter++
|
+-- Read
|
+-- Add 1
|
+-- Write

```

With:

```
AtomicInteger counter =  
    new AtomicInteger(0);  
  
counter.incrementAndGet();
```

the update is performed using an atomic operation.

```
incrementAndGet()
```

|

v

Atomic update

|

v

New value

10. AtomicBoolean

AtomicBoolean provides atomic operations on boolean values.

```
import java.util.concurrent.atomic.AtomicBoolean;
```

```
public class AtomicTypes {  
  
    public static void main(String[] args) {
```

```
        AtomicBoolean running =  
            new AtomicBoolean(false);
```

```
        System.out.println(  
            "Initial: " + running.get()  
        );
```

```
        running.set(true);
```

```
        System.out.println(  
            "Updated: " + running.get()  
        );
```

```
    }  
}
```

Output

```
Initial: false  
Updated: true
```

11. compareAndSet()

One of the most important atomic operations is **Compare-And-Set (CAS)**.

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicTypes {

    public static void main(String[] args) {

        AtomicInteger value =
            new AtomicInteger(10);

        boolean updated =
            value.compareAndSet(10, 20);

        System.out.println(
            "Updated: " + updated
        );

        System.out.println(
            "Value: " + value.get()
        );
    }
}
```

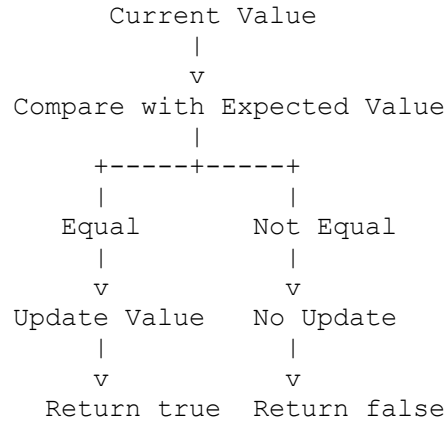
Output

```
Updated: true
Value: 20
```

The operation means:

```
Current value == 10?
      |
    +---+---+
      |       |
    Yes      No
      |       |
      v       v
Set 20      No update
      |
      v
Return true
```

12. CAS Flow



This **Compare-And-Set** mechanism is a fundamental building block for many lock-free and non-blocking algorithms.

13. AtomicLong Example

```
import java.util.concurrent.atomic.AtomicLong;

public class AtomicTypes {

    public static void main(String[] args) {

        AtomicLong counter =
            new AtomicLong(100000);

        counter.addAndGet(5000);

        System.out.println(
            "Counter: " + counter.get()
        );
    }
}
```

Output

Counter: 105000

14. AtomicReference

AtomicReference<T> can atomically update an object reference.

```
import java.util.concurrent.atomic.AtomicReference;

public class AtomicTypes {

    public static void main(String[] args) {

        AtomicReference<String> name =
            new AtomicReference<String>("Anurag");

        System.out.println(
            "Before: " + name.get()
        );

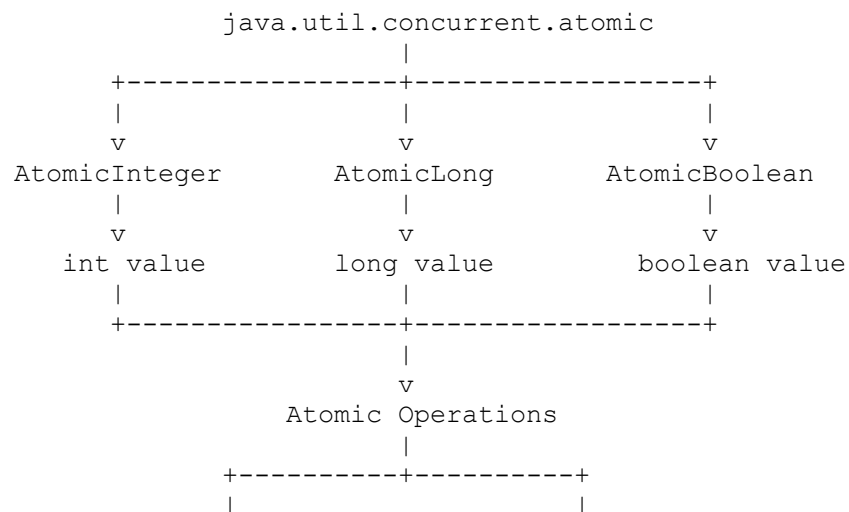
        name.set("Rahul");

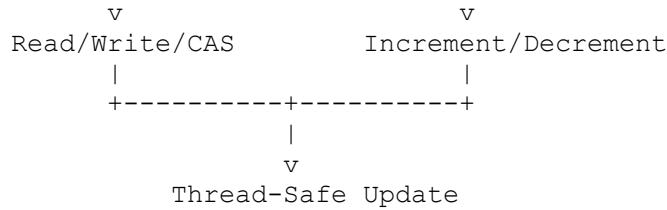
        System.out.println(
            "After: " + name.get()
        );
    }
}
```

Output

Before: Anurag
After: Rahul

15. Atomic Types Architecture





16. Common Atomic Classes

Class	Purpose
AtomicInteger	Atomic operations on int
AtomicLong	Atomic operations on long
AtomicBoolean	Atomic operations on boolean
AtomicReference<V>	Atomic operations on object references
AtomicIntegerArray	Atomic operations on int[]
AtomicLongArray	Atomic operations on long[]
AtomicReferenceArray<E>	Atomic operations on object-reference arrays

17. Real-World Example — Request Counter

A web application might maintain a concurrent request counter.

```
import java.util.concurrent.atomic.AtomicInteger;
```

```
public class AtomicTypes {
```

```
    private static AtomicInteger requestCount =  
        new AtomicInteger(0);
```

```
    public static void recordRequest() {  
        requestCount.incrementAndGet();  
    }
```

```
    public static void main(String[] args) {  
        recordRequest();  
    }
```

```

        recordRequest();
        recordRequest();

        System.out.println(
            "Total Requests: "
            + requestCount.get()
        );
    }
}

```

Output

Total Requests: 3

In a multithreaded server, multiple threads can safely update the counter using `incrementAndGet()`.

18. Atomic Types vs synchronized

Atomic classes and traditional synchronization solve related concurrency problems, but they are not interchangeable in every situation.

Atomic approach

```

AtomicInteger counter =
    new AtomicInteger();

counter.incrementAndGet();

```

Synchronized approach

```

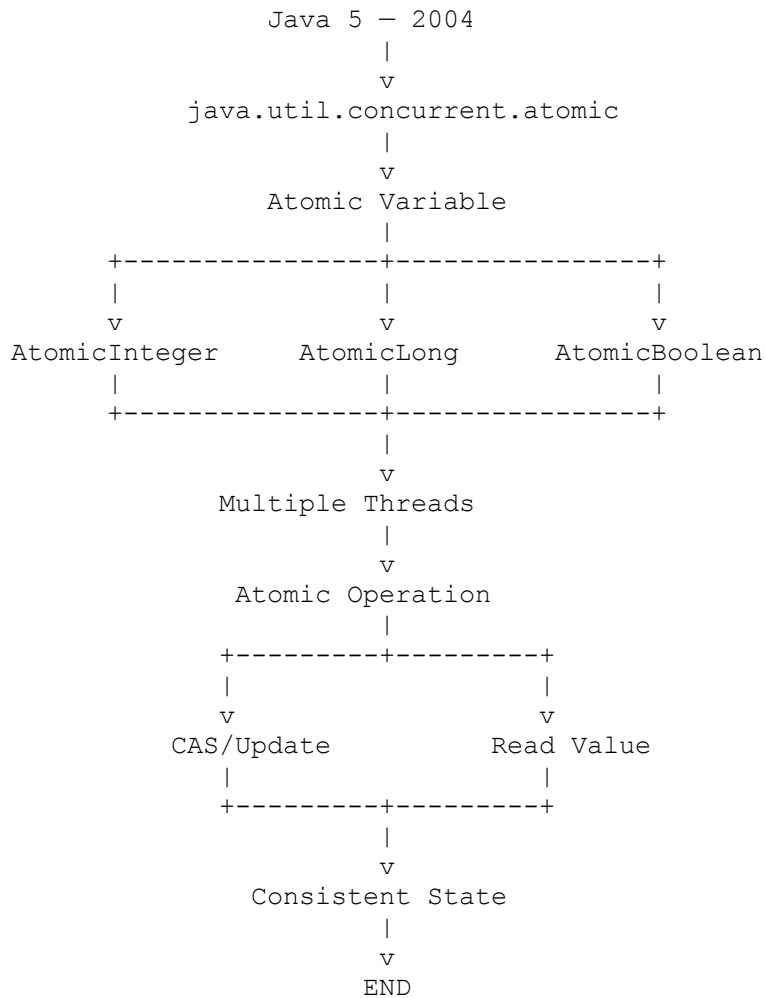
synchronized (this) {
    counter++;
}

```

Atomic Types	synchronized
Good for individual atomic variables	Good for protecting larger critical sections
Commonly uses CAS-based operations	Uses monitor-based synchronization
No explicit lock needed for basic operations	Requires acquiring a monitor
Useful for counters and state flags	Useful for compound operations involving multiple shared states

Non-blocking for many operations	Threads can block waiting for a monitor
---	---

19. Complete Flowchart



20. Key Points

- **Introduced:** Java 5
- **Year:** 2004
- **Package:** java.util.concurrent.atomic
- Provides thread-safe operations on individual variables.
- Important classes include:
 - AtomicInteger
 - AtomicLong
 - AtomicBoolean

- AtomicReference
- Supports operations such as:
 - get()
 - set()
 - incrementAndGet()
 - decrementAndGet()
 - addAndGet()
 - compareAndSet()
- Atomic operations are useful for **counters, flags, state variables, and concurrent algorithms**.
- Many atomic update operations use **compare-and-set (CAS)** techniques.
- Atomic classes do not replace synchronized for every concurrency problem; they are particularly useful for atomic updates to individual variables.

In one sentence

Java atomic types, introduced in Java 5, provide thread-safe, atomic operations on individual variables such as integers, longs, booleans, and object references, enabling efficient concurrent programming without explicit locking for many common operations.